

Chapter 14. Data Cleaning & Preparation

Profile, fix, validate, and reconcile messy data in Power Query, SQL, and pandas, and prove the result to the rupee.

Approved chapter v1.2

Contents

Chapter 14. Data Cleaning & Preparation

Why this matters

In plain English

14.1 A cleaning workflow you can repeat

14.2 Profiling a new dataset

14.3 Missing values

14.4 Duplicates, exact and fuzzy

14.5 Inconsistent categories and typos

14.6 Outliers: error or reality?

14.7 Dates, time zones, units, and currency

14.8 Joining messy sources

14.9 Validation rules: checks that must return zero

14.10 Reconciling and documenting every decision

14.11 The whole pipeline in Excel and Power Query

14.12 Running this chapter in MySQL

14.13 A first look at cleaning in pandas

Common mistakes and how to spot them

In the real world: the branch league table

Tools

The project: clean Riverstone's operations export and write a data-quality report

Timed challenge: the CRM clean-up sprint (40 minutes)

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 14. Data Cleaning & Preparation

Part II — The Analyst

Chapter at a glance

You will learn to: follow a repeatable cleaning workflow (load, profile, fix, validate, reconcile, document) · profile an unfamiliar dataset in minutes with counts, patterns, and ranges · decide what to do with missing values: standardize, repair, quarantine, or keep and label · find exact and fuzzy duplicates without merging the wrong records · standardize categories and typos with mapping tables · judge whether an outlier is an error or reality · parse mixed date formats, reject impossible dates, convert time zones, and fix units and currency text · repair join keys and measure match rates before joining messy sources · write validation rules that must return zero · reconcile a cleaned table to its source to the rupee · keep a cleaning log that someone else can audit · do all of it in Excel and Power Query, in SQL (PostgreSQL and MySQL), and in a first look at pandas.

Before you start: Chapter 1 (section 1.10, data quality), Chapter 10 (importing CSV files, text and date functions), Chapter 11 (Power Query), and Chapters 12–13 (SQL, CTEs, window functions).

Time needed: 20–25 hours of reading and practice, spread over three weeks.

Tools: Excel for Windows (Microsoft 365) with Power Query, or Google Sheets for the spreadsheet parts; PostgreSQL 16 and MySQL 8.0 with DBeaver or MySQL Workbench; Python 3.12 with pandas for section 14.13's preview.

Practice data: the full Riverstone dataset from this chapter onward (`companion/full/` , about 5,000 customers and 209,006 order lines for 2023–2025, described in `companion/full/DATA_SPEC.md`) and two messy exports built from it (`companion/ch14/`): `orders_q4_2025_export.csv` , the ERP's order lines for October–December 2025 as Riverstone's four branch sales offices enter them, and `customers_crm_export.csv` , the CRM's customer list. Because the exports were made by damaging real data in documented ways, you can check every cleaning step against the truth: the clean result must match the database line for line.

Why this matters

Ask analysts where their time goes and many will say "cleaning". Whatever the exact share in your job, the reason is always the same: data is produced by people and systems that aren't thinking about your analysis. A branch office types dates its own way. A CRM lets two salespeople enter the same customer. An export tool writes a page header every few thousand rows. None of that is anyone's fault, and all of it lands on the analyst.

Cleaning isn't glamorous, but it's where trust is won or lost. In Chapter 1, a customer list with three spellings of Mumbai turned "how many customers are in Mumbai?" into a wrong answer with no error message. In Chapter 10, a CSV opened with the wrong settings shifted 135 dates by weeks. In Chapter

11, a month-end workbook was ₹64,596 off because of a short range and a double paste. Every one of those was a cleaning problem that someone didn't catch.

This chapter turns cleaning from a pile of fixes into a **method**. You'll clean a real-sized export (25,976 rows entered by four branch sales offices, each in its own way) with more than a dozen kinds of planted problems, in three tools, and prove the result is right by reconciling it with the source. The same method works on any dataset you'll meet: a sales export, a survey, a supplier price list, or a machine log.

It also matters in interviews. "Here's a messy file; what would you check first?" is one of the most common analyst interview questions, and the profiling checklist in section 14.2 is a strong answer.

In plain English

Picture a school office on results day. Marks arrive from twelve teachers. One writes dates as 05/11, another as 11-05. One enters "absent", another leaves the box blank, a third writes 0. Two sheets for Class 8B were photocopied twice. Somebody typed 950 instead of 95 for one student.

A careful office clerk doesn't start adding marks. First she **looks over every sheet** (profiling): how many students, which formats, any strange values. Then she **fixes what she can prove** (cleaning): rewrites dates one way, removes the photocopied duplicates. For the 950, she **checks with the teacher** rather than guessing (quarantine). She **keeps a note** of every change (the cleaning log). Before the results go to the principal, she **checks the totals against the class registers** (reconciliation), because a result sheet that looks tidy can still be wrong.

Here's the mapping:

- Looking over every sheet is **profiling**: counting rows, values, blanks, patterns, and ranges.
- A blank, "absent", and 0 are the **missing values** problem: three ways of recording "we don't know", and one of them (0) is dangerously wrong.
- Photocopied sheets are **duplicates**; the same student written as "Rahul M." and "RAHUL MEHTA" is a **fuzzy duplicate**.
- Rewriting every "absent" and "Abs." the same way is **standardizing categories** with a **mapping table**.
- The 950 is an **outlier** that's probably an error; a brilliant 99 is an outlier that's real.
- The class register check is **reconciliation**; the note of every change is the **cleaning log**.

14.1 A cleaning workflow you can repeat

Most cleaning goes wrong in one of two ways: people fix problems in whatever order they notice them (and miss the ones they don't notice), or they fix values by overwriting the original data (and can't undo or explain it later). A workflow prevents both.

The cleaning workflow, with Riverstone's Q4 2025 order export

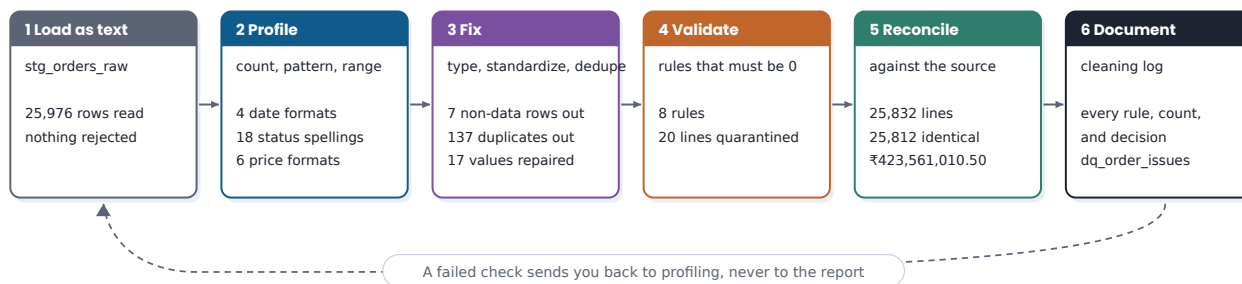


Figure 14.1 – The cleaning workflow applied to Riverstone's Q4 2025 order export. Every number comes from queries in this chapter.

1. **Load as text.** Bring the raw data in without letting any tool guess types (Chapter 10, section 10.4). A staging table where every column is text accepts anything, so nothing is silently rejected or converted.
2. **Profile.** Count rows, distinct values, blanks, patterns, and ranges for every column *before* changing anything. Write down what you find.
3. **Fix.** Convert types, standardize categories, remove duplicates, and repair what can be proven, **in a new table or query**, never in the source.
4. **Validate.** Run rules that must return zero failures ("every status is one of four values", "every customer code exists"). Anything that fails is fixed, or quarantined with a reason.
5. **Reconcile.** Compare the cleaned result with an independent total: the source system, finance, or last month's approved report.
6. **Document.** Keep a cleaning log: each rule, how many rows it affected, and what was decided.

Two principles sit underneath:

- **Never clean the source.** Chapter 13 said it for SQL: don't delete from the system of record as part of an analysis. The same goes for a shared spreadsheet or a CSV someone else produced. Clean into a new table, a query, or a copy.
- **Cleaning must be repeatable.** Next quarter's export will have the same problems. A recorded Power Query, a SQL script, or a Python script cleans it again in seconds; a morning of manual edits in a spreadsheet has to be redone by hand.

The data for this chapter

From this chapter on, Riverstone's examples use the **full dataset** (`companion/full/`): 5,027 customer records, 116,194 orders, and 209,006 order lines from 2023 to 2025, loaded as the databases `riverstone_full` in PostgreSQL and MySQL. It's the whole company, not only the 24 key accounts of Chapters 10–13. Those 24 customers are inside it, with exactly the same 2025 orders, so their ₹4,335,471 still holds, but Riverstone's total 2025 net revenue is **₹1,146,641,651** (₹114.7 crore).

Chapter 14 cleans two exports made from that database:

File	What it is	Rows	Problems planted
orders_q4_2025_export.csv	Order lines for October–December 2025, exported from the ERP with each branch sales office's data-entry habits intact	25,976 (plus a header)	page headers and a footer, duplicate rows, four date formats, impossible dates, codes without leading zeros, prices as currency text, discounts as fractions, quantities in cartons, typed extra zeros, blanks, 18 spellings of status, 17 spellings of branch, trailing spaces, UTC timestamps
customers_crm_export.csv	The CRM's customer list	5,027	duplicate customers with name variants, city spellings and old names, placeholder text for missing values, segment spellings, invalid emails, two date formats, signup dates in the 2060s

Riverstone runs one ERP, the system of record from Chapter 3. Orders reach it through four **branch sales offices**: Mumbai HO, Bengaluru, Delhi, and Kolkata. (They're sales offices, not the plants at Taloja and Chakan or the Bhiwandi warehouse.) Each office enters orders a little differently. Kolkata's office was moved over from an older billing system and still loads its orders into the ERP through a spreadsheet upload, which writes year-first dates, drops leading zeros from customer codes, and stores discounts as fractions. Delhi enters some quantities in cartons. Mumbai HO and Bengaluru paste prices as text. One system, four sets of habits: that's what the export reflects.

The truth is known: after cleaning, the order export must match the database's Q4 2025 order lines exactly. `answer_key.json` lists every planted problem with its count. Don't open it until you've profiled the files yourself.

Loading the exports as text

SQL. The companion scripts create staging tables where every column is text, then load the CSVs:

```
psql -d riverstone_full -f sql/ch14_load_postgresql.sql
mysql --local-infile=1 -u root -p riverstone_full < sql/ch14_load_mysql.sql
```

Run them from `companion/ch14`. They create `stg_orders_raw` and `stg_customers_raw`, plus four small mapping tables used in section 14.5. The table definition shows the idea:

```
CREATE TABLE stg_orders_raw (
  order_item_id TEXT, order_id TEXT, order_date TEXT, customer_code TEXT, product_id TEXT, quantity
  TEXT,
  qty_unit TEXT, unit_price TEXT, discount_pct TEXT, status TEXT, sales_rep TEXT, branch TEXT,
  entered_at_utc TEXT);
```

(MySQL uses `VARCHAR` lengths instead of `TEXT`, and `LINES TERMINATED BY '\r\n'`, because the export has Windows line endings.)

Excel and Power Query. **Data → Get Data → From File → From Text/CSV**, choose the file, set **Data Type Detection** to **Do not detect data types**, and click **Transform Data**. Every column arrives as text. Delete any automatic **Changed Type** step (Chapter 11, section 11.7).

Google Sheets. **File** → **Import** → **Upload**, clear **Convert text to numbers, dates, and formulas**. At 25,976 rows × 13 columns the order export fits comfortably within Sheets' limits, but formulas over it will be slow; for this chapter's spreadsheet work in Sheets, use a filtered copy (for example, one branch).

14.2 Profiling a new dataset

Profiling means describing a dataset before you use it: how big it is, what each column contains, and what looks wrong. Twenty minutes of profiling saves days of rework, and it tells you which cleaning steps you'll need.

The profiling checklist

For the table as a whole:

1. **How many rows, and what is one row?** (the grain, Chapter 1)
2. **Is the key unique?** Compare the row count with the count of distinct IDs.
3. **Are all rows data?** Look for headers, totals, and notes mixed in.

For each column:

4. **Blanks and placeholders:** how many empty values, and which text stands in for "missing" (N/A , - , unknown , NULL)?
5. **Distinct values:** for categories, list every value with its count.
6. **Patterns:** for codes, dates, and numbers stored as text, replace every digit with 9 and every letter with A, then count the patterns.
7. **Ranges:** for numbers and dates, the minimum and maximum. Are they possible?
8. **Relationships:** do codes match the lists they should match (customers, products)?

Profiling the order export in SQL

Rows and key.

```
SELECT COUNT(*) AS rows_loaded, COUNT(DISTINCT order_item_id) AS distinct_ids FROM stg_orders_raw;
```

```
rows_loaded | distinct_ids
-----+-----
      25976 |         25833
(1 row)
```

Order lines should have one row each, but there are 143 more rows than distinct IDs. Something is repeated, or some rows aren't order lines at all.

Rows that aren't data. A real order line ID is all digits:


```
SELECT order_item_id, COUNT(*) AS rows FROM stg_orders_raw WHERE order_item_id !~ '^[0-9]+$' OR
order_item_id IS NULL GROUP BY order_item_id ORDER BY rows DESC;
```

```
order_item_id | rows
-----+-----
order_item_id |    6
              |    1
(2 rows)
```

Six rows contain the word `order_item_id`: the export repeats its **header row** every 4,000 lines, a leftover from a paginated report. One row has an empty ID: the **footer** (Report generated 01-01-2026 02:00; rows: 25975). That accounts for one of the 25,833 "distinct IDs" too. Removing these 7 rows leaves 25,969 data rows, still 137 more than distinct IDs: candidates for duplicates (section 14.4).

Watch out: footers lie too. The footer claims 25,975 rows. That count includes the six repeated headers and the duplicate rows, so it's the number of lines the export tool wrote, not the number of order lines. Never use a file's own summary line as your reconciliation total.

Date patterns. Replacing every digit with `9` shows the shapes of the values without their details:

```
SELECT regexp_replace(order_date, '[0-9]', '9', 'g') AS date_pattern, COUNT(*) AS rows
FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$' GROUP BY 1 ORDER BY rows DESC;
```

```
date_pattern | rows
-----+-----
99-99-9999   | 20890
9999-99-99   | 3227
99/99/9999   | 1812
99999        | 40
(4 rows)
```

Four formats in one column. `99-99-9999` is Riverstone's usual day-first format. `9999-99-99` is year-first. `99/99/9999` uses slashes. And `99999` is five digits with no separators: an **Excel date serial number** (Chapter 10, section 10.3) from someone who pasted dates as values before exporting. A pattern profile can't tell whether `05-11-2025` is 5 November or 11 May; section 14.7 settles that.

Categories. List every status with its count:

```
SELECT status, COUNT(*) AS rows FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$' GROUP BY
status ORDER BY rows DESC;
```


status	rows
Delivered	19835
Pending	1026
Shipped	1004
Cancelled	985
DELIVERED	697
Dlvd	694
Delivered	672
delivered	653
Pending	57
pending	52
Shipped	51
shipped	45
SHIPPED	44
PENDING	39
Cxl	32
CANCELLED	31
Canceled	28
cancelled	24

(18 rows)

Eighteen values for a field with four real statuses. `Delivered` appears twice because one version has a trailing space you can't see. `Dlvd` and `Cxl` are abbreviations, and `Canceled` is the American spelling. A report that filters `status = 'Cancelled'` would miss 115 cancelled lines.

Blanks. Count empty values per column:

```
SELECT COUNT(*) FILTER (WHERE product_id IS NULL OR product_id = '') AS product_id,
       COUNT(*) FILTER (WHERE quantity IS NULL OR quantity = '') AS quantity,
       COUNT(*) FILTER (WHERE sales_rep IS NULL OR sales_rep = '') AS sales_rep,
       COUNT(*) FILTER (WHERE branch IS NULL OR branch = '') AS branch
FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$';
```

product_id	quantity	sales_rep	branch
8	14	817	0

(1 row)

`FILTER (WHERE ...)` counts only the rows that meet its condition (PostgreSQL; section 14.12 gives the MySQL form). Checking both `IS NULL` and `= ''` matters: PostgreSQL's CSV loader turns an unquoted empty field into NULL, MySQL's turns it into an empty string, and a spreadsheet shows both as a blank cell.

Numbers stored as text. The pattern trick works for prices too:

```
SELECT regexp_replace(unit_price, '[0-9]', '9', 'g') AS price_pattern, COUNT(*) AS rows
FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$' GROUP BY 1 ORDER BY rows DESC;
```

price_pattern	rows
999	22392
9999	1261
Rs. 999	1137
₹999.99	1052
Rs. 9,999	68
₹9,999.99	59

(6 rows)

Most prices are plain numbers, but 2,316 rows carry currency text: Rs. 430 , ₹1,400.00 . None of them can be summed until they're converted, and a spreadsheet would count them as text (Chapter 10).

Discounts. Listing distinct values shows the problem immediately:

```
SELECT discount_pct, COUNT(*) AS rows FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$' GROUP BY 1 ORDER BY 1;
```

discount_pct	rows
0	14711
0.05	878
0.08	164
0.10	150
0.12	179
10	1250
12	1232
5	6129
8	1276

(9 rows)

Riverstone's discounts are 0, 5, 8, 10, or 12 percent. Some rows record 5% as 0.05 , a **fraction** instead of a percentage. Applied as written, 0.05 would mean a discount of five hundredths of a percent. (The values also sort as text, which is why 10 comes before 5 .)

Quantities and units.

```
SELECT qty_unit, COUNT(*) AS rows, MIN(NULLIF(quantity,'')::int) AS min_qty,
MAX(NULLIF(quantity,'')::int) AS max_qty
FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$' GROUP BY qty_unit;
```

qty_unit	rows	min_qty	max_qty
PCS	25828	5	700
CTN	141	1	9

(2 rows)

Most quantities are in pieces (PCS), but 141 rows are in cartons (CTN) of 10 pieces. And a maximum of 700 pieces on one line is far above anything Riverstone normally sells; section 14.6 investigates.

Code lengths.

```
SELECT LENGTH(customer_code) AS code_length, COUNT(*) AS rows FROM stg_orders_raw WHERE
order_item_id ~ '^[0-9]+$' GROUP BY 1 ORDER BY 1;
```

```
code_length | rows
-----+-----
          2 |    40
          3 |   497
          4 | 25432
(3 rows)
```

Customer codes are four digits (0124). 537 rows have shorter codes: leading zeros lost somewhere upstream (Chapter 10's most common import accident).

What the profile tells you

In about ten queries, the profile found problems in almost every column, and it did so **before** any number was calculated. Write the findings down as a list; it becomes the plan for sections 14.3 to 14.8 and, later, the cleaning log.

Column	Finding	Planned action
(whole file)	6 repeated headers, 1 footer	Remove non-data rows
(whole file)	137 more data rows than distinct IDs	Investigate duplicates
order_date	4 formats incl. Excel serials	Parse each format explicitly
customer_code	537 codes shorter than 4 digits	Pad with leading zeros; check match rate
product_id , quantity	8 and 14 blanks	Decide: repair, quarantine, or keep
quantity , qty_unit	cartons; a maximum of 700	Convert units; investigate outliers
unit_price	2,316 values with currency text	Strip text safely
discount_pct	some fractions	Convert to percentages
status , branch	18 and 17 spellings	Map to standard values
sales_rep	817 blanks, trailing spaces	Trim; keep blanks as unknown

Profiling in Excel and Power Query

Power Query has a built-in profiler. In the Power Query Editor, turn on **View → Data Preview → Column quality, Column distribution, and Column profile**. At the bottom of the window, change **Column profiling based on top 1000 rows** to **Column profiling based on entire data set**; the default profiles only the first thousand rows, which would miss every Kolkata row and most of the problems here.

- **Column quality** shows the percentage of **Valid**, **Error**, and **Empty** values under each header.
- **Column distribution** shows the number of **distinct** and **unique** values (unique = appearing exactly once).
- **Column profile** (click a column) shows count, errors, empty, distinct, min, max, and a value distribution chart. On `status` it lists the 18 spellings; on `order_date` , the range tells you nothing (it's text), which is itself a finding.

In a plain worksheet, the same checks are formulas you already know: `=ROWS()` and `=COUNTA(UNIQUE(A2:A25977))` for rows and distinct IDs (Chapter 11, section 11.6), a pivot table with `status` in both Rows and Values (Count) for the categories, `=COUNTBLANK()` for blanks, and `=MIN()` / `=MAX()` for ranges. For patterns, a helper column with nested `SUBSTITUTE` s works, or in Microsoft 365, `=REGEXREPLACE(C2, "[0-9]", "9")` where available.

Tool note: profile the whole file. Power Query's "top 1000 rows" default, Excel's filter drop-down (which lists at most 10,000 distinct values), and a quick scroll through the first screen all look at part of the data. Problems in one branch or one month often sit far down the file. Always state what your profile covered.

14.3 Missing values

A **missing value** is a cell with no usable value. It sounds simple, but the first job is to recognize all the ways "missing" is written, and the second is to decide what the gap means.

Recognize every form of missing

The CRM export shows the usual suspects in the `city` column:

```
SELECT COALESCE(city, '(NULL)') AS city_as_exported, COUNT(*) AS customers FROM stg_customers_raw
WHERE city IS NULL OR LOWER(TRIM(city)) IN ('', 'n/a', '-', 'unknown', 'null') GROUP BY 1 ORDER BY
2 DESC;
```

city_as_exported	customers
NULL	24
N/A	22
-	20
unknown	19
(NULL)	15

(5 rows)

Five different records of the same fact, "we don't know this customer's city", for 100 customers. Two of them deserve a closer look:

- **NULL (24 customers)** is the four-letter **text** `NULL`, written into the CSV by a system that printed its missing values as words. It is not a database `NULL`, so `WHERE city IS NULL` doesn't find it.
- **(NULL) (15 customers)** is the query's label for a real, empty value.

Standardize all of them to a real `NULL` (or a blank cell in a spreadsheet) before anything else. The mapping table in section 14.5 does it: `n/a`, `-`, `unknown`, and `null` all map to `NULL`.

Watch out: 0 is not missing. A blank quantity and a quantity of 0 mean different things. Replacing blanks with 0 (a common "fix" in spreadsheets, and the default of some tools) turns "we don't know" into "we sold nothing", and every total and average that includes those rows is wrong without any visible gap.

Why is it missing?

Statisticians describe three kinds of missingness, and the idea behind them is practical even without the formal names:

Kind	Meaning	Riverstone example	Consequence
Missing completely at random	Gaps have nothing to do with the data	a few quantities lost in a system glitch	Excluding them loses precision but doesn't bias results
Missing at random (given other columns)	Gaps depend on something you can see	sales rep missing more often in one branch	Look at the gap by that column before trusting totals
Missing not at random	Gaps depend on the missing value itself	customers with low satisfaction skip the survey question	Averages of what remains are biased; say so

Always look at missingness by a few important columns before deciding. For the sales rep:

```
SELECT branch, COUNT(*) AS lines, COUNT(*) FILTER (WHERE sales_rep IS NULL) AS no_rep,  
       ROUND(100.0 * COUNT(*) FILTER (WHERE sales_rep IS NULL) / COUNT(*), 1) AS pct  
FROM clean_order_lines GROUP BY branch ORDER BY branch;
```

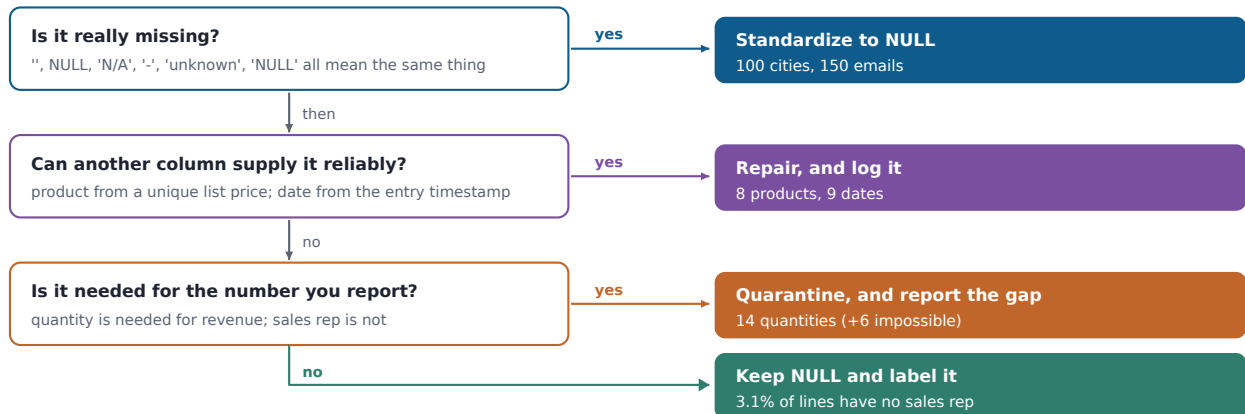
```
branch | lines | no_rep | pct  
-----+-----+-----+-----  
Bengaluru | 7185 | 220 | 3.1  
Delhi | 6274 | 213 | 3.4  
Kolkata | 3210 | 91 | 2.8  
Mumbai H0 | 9163 | 288 | 3.1  
(4 rows)
```

(This query runs on `clean_order_lines`, the cleaned table the companion script builds, so the branches are already standardized and duplicates removed; sections 14.4–14.7 explain each step.) About 3% of lines have no rep in every branch. That looks like a steady process gap (orders entered without assigning a rep), not a problem concentrated in one place. A rep-level report can still be produced, with an "(unassigned)" row; a branch head asking "why is my team's number low?" can see that the gap is the same everywhere.

Four decisions

Figure 14.2 gives a decision path for each missing (or clearly wrong) value.

What to do with a missing or wrong value



Never fill an identifier, a price, or a quantity with 0 or an average: the report would look complete and be wrong.

Figure 14.2 — What to do with a missing or wrong value, with the Riverstone counts from this chapter.

1. **Standardize.** Every form of missing becomes one form (NULL). No judgment needed.
2. **Repair,** when another column can supply the value **reliably**, and log the repair.
 - Eight order lines have a blank `product_id`. In 2025, every product had a different list price (₹115, ₹290, ₹380, ₹430, ₹620, ₹750, ₹1,150, ₹1,400), so the price identifies the product with certainty. The repair is safe *for this data*; if two products shared a price it wouldn't be.
 - Nine lines have impossible dates. Every line also has an entry timestamp, and the entry date in Indian time is always the order date. That's a reliable source for a repair (section 14.7).
3. **Quarantine,** when the value is needed for the number you report and can't be recovered. Fourteen lines have a blank quantity. Revenue can't be calculated for them, and there's no honest way to guess. Move them to a **quarantine** list with the reason, exclude them from revenue, and **report the gap**: "14 lines (₹X of list value, to be confirmed by the ERP team) are excluded."
4. **Keep and label,** when the value isn't needed for the result. A missing sales rep doesn't stop revenue from being counted; it only means the line can't be credited to a person.

Simplification note: imputation. Statistical and machine-learning work sometimes **imputes** missing values: fills them with a median, a group average, or a model's prediction. It can be the right call for building a model (Part IV covers it). For business reporting, where each line is a real transaction, don't invent values. Quarantine and report instead.

Missing values in each tool

Task	Excel / Power Query	SQL	pandas (section 14.11)
Count blanks	COUNTBLANK ; Power Query Column quality	COUNT(*) FILTER (WHERE col IS NULL OR col = '')	df[col].isna().sum()
Placeholders to blank	Transform → Replace Values (N/A → empty)	NULLIF(TRIM(col), '') , or a mapping table	df[col].replace(["N/A", "-"], None)
Keep and label	=IF(A2="", " (unassigned)", A2) ; Replace Values null → (unassigned) for display	COALESCE(sales_rep, '(unassigned)')	df[col].fillna(" (unassigned)")

Task	Excel / Power Query	SQL	pandas (section 14.11)
Remove rows	Home → Remove Rows → Remove Blank Rows (whole row blank only); filter a column	WHERE quantity IS NOT NULL	df.dropna(subset=["quantity"])
Fill down (group labels)	Transform → Fill → Down	LAST_VALUE(...) IGNORE NULLS is limited; use a window with COUNT (Chapter 13)	df[col].ffill()

Fill down deserves a note. Reports exported "for printing" often show a branch or month name only on the first row of each group and leave the rows below it blank. Those blanks aren't missing; they mean "same as above", and **Fill Down** restores them. Using it on a column where blank really means unknown (such as `sales_rep`) would copy one person's name onto hundreds of orders they never touched.

14.4 Duplicates, exact and fuzzy

A **duplicate** is a record that describes the same real thing as another record. Exact duplicates are identical in every column; **fuzzy duplicates** refer to the same thing but are written differently.

Exact duplicate rows

```
SELECT COUNT(*) AS data_rows, COUNT(DISTINCT stg_orders_raw.*) AS distinct_rows FROM stg_orders_raw
WHERE order_item_id ~ '^[0-9]+$';
```

```
data_rows | distinct_rows
-----+-----
      25969 |          25832
(1 row)
```

`COUNT(DISTINCT stg_orders_raw.*)` counts distinct whole rows (PostgreSQL). There are **137** rows that are exact copies of another row. Which IDs?

```
SELECT order_item_id, COUNT(*) AS copies FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$' GROUP
BY 1 HAVING COUNT(*) > 1 ORDER BY 1 LIMIT 5;
```

```
order_item_id | copies
-----+-----
      184028   |      2
      184256   |      2
      184480   |      2
      184509   |      2
      184594   |      2
(5 rows)
```

Each duplicate appears exactly twice, and both copies are identical in every column. That's the signature of an **export glitch** (a page of results written twice), not two real sales. It's safe to keep one copy: `SELECT DISTINCT *`, Power Query's **Home → Remove Rows → Remove Duplicates** with all

columns selected, Excel's **Data → Remove Duplicates** (all columns ticked), or `drop_duplicates()` in pandas.

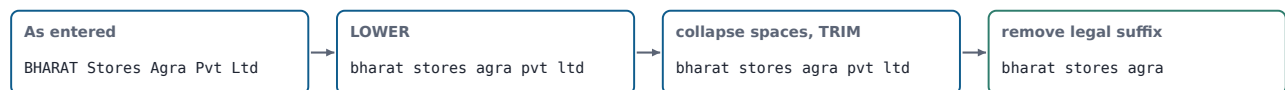
Watch out: same ID, different values. Before removing duplicates by ID alone, check whether rows with the same ID are really identical. If two rows share `order_item_id` 184028 but have different quantities, one of them was *updated*, and you need the latest version, not the first one you meet. `COUNT(DISTINCT stg_orders_raw.*)` compared with `COUNT(DISTINCT order_item_id)` answers the question: here both give 25,832, so every repeated ID is a true copy.

When duplicates are real. Two identical lines can be legitimate: a customer who orders the same product twice on the same day in two separate orders has two rows that differ only in their IDs. That's why you check duplicates on the **key** (here `order_item_id`), and why you never remove "duplicates" on a subset of columns without asking what one row represents.

Fuzzy duplicate customers

The CRM export has 5,027 customer records. Riverstone's customer team suspects some businesses were entered twice. An exact match on the name finds only 9 pairs: the other duplicates differ in capitals, spaces, or a legal suffix. The fix is a **match key**: a normalized version of the name used only for finding candidates.

Building a match key for customer names



match on name key only	48 groups, 96 records	every planted duplicate found; 0 false matches
match on name key + city	46 groups, 92 records	misses 2 pairs where one record has no city
match on exact name	9 groups	misses every case, space, and suffix variant

Riverstone's CRM export: 5,027 records, 48 planted duplicates. A key finds candidates; a person confirms them before any merge.

Figure 14.3 – A match key removes differences that don't matter (case, extra spaces, a legal suffix) so duplicates can be found. The choice of matching columns changes what you find.

The cleaning script builds `match_key` in the `clean_customers` table:

```
LOWER(regex_replace(regex_replace(customer_name, '\s+(pvt\.\?\s*\ltd\.\?|private limited)$', '', 'i'), '\s+', ' ', 'g')) AS match_key
```

Reading from the inside out: remove a trailing "Pvt Ltd" (with or without dots) or "Private Limited", case-insensitively; replace every run of spaces with one space; convert to lower case. (The name has already been trimmed.) Then group by the key:

```
SELECT match_key, COUNT(*) AS records, STRING_AGG(customer_code || ' ' || customer_name, ' | ' ORDER BY customer_code) AS versions
FROM clean_customers GROUP BY match_key HAVING COUNT(*) > 1 ORDER BY match_key LIMIT 5;
```

match_key	records	versions
-----	-----	-----
balaji distributors	2	2278 Balaji Distributors 5010 BALAJI DISTRIBUTORS
bharat general store thane	2	1620 Bharat General Store Thane 4982 Bharat General Store Thane
bharat logistics delhi	2	4090 Bharat Logistics Delhi 4996 Bharat Logistics Delhi
bharat silk route home needs	2	3874 Bharat Silk Route Home Needs 5023 Bharat Silk Route Home Needs Pvt Ltd
bharat stores agra	2	2153 Bharat Stores Agra 5020 BHARAT STORES AGRA
(5 rows)		

```
SELECT COUNT(*) AS duplicate_groups FROM (SELECT match_key FROM clean_customers GROUP BY match_key
HAVING COUNT(*) > 1) g;
```

duplicate_groups

48
(1 row)

48 groups. Checked against the answer key, all 48 are real planted duplicates, and none is a false match. (bharat general store thane looks identical in both versions because the difference is a double space that the display collapses.)

A natural refinement is to match on name **and city**, to avoid merging two different "Balaji Distributors" in different cities. On this data that finds only 46 groups: two duplicate pairs have a city on one record and none on the other, and NULL never equals anything (Chapter 12). Every matching rule is a trade-off between missing real duplicates and merging different businesses. Measure both kinds of error when you can, and have a person confirm candidates before any merge.

Real fuzzy matching

Match keys handle predictable differences. Typos ("Balaji Distrubutors") and word order ("Stores Balaji") need **similarity** measures:

- **PostgreSQL:** the `pg_trgm` extension compares strings by shared three-letter sequences. After `CREATE EXTENSION pg_trgm;`, `similarity('Metro Mart', 'METRO MART Pvt Ltd')` returns 0.5555556 (1 means identical). Candidates above a threshold such as 0.6, within the same city, go to a review list.
- **MySQL:** `SOUNDEX()` matches similar-sounding English words, which is crude for Indian business names; fuzzy matching usually moves to Python or a dedicated tool.
- **Power Query: Home → Merge Queries with Use fuzzy matching to perform the merge**, and under **Fuzzy matching options** set a **Similarity threshold** (0 to 1), **Ignore case**, **Match by combining text parts** (ignores spaces), and a **Transformation table** for known equivalents (`Pvt Ltd` → empty). Power Query uses the Jaccard similarity algorithm.
- **Excel's Fuzzy Lookup add-in** from Microsoft does the same in older versions; **pandas** users reach for libraries such as `rapidfuzz` (Chapter 18).

Watch out: never auto-merge customers. Merging two customer records moves order history, credit limits, and contacts. A wrong merge is much harder to undo than a missed duplicate. Cleaning for **analysis** can group candidates with a match key in your query; cleaning the **CRM itself** needs a review list, an owner's sign-off, and the CRM's own merge feature.

14.5 Inconsistent categories and typos

A **category** (status, branch, city, segment) should have a small, fixed set of allowed values. Real data has many spellings of each. The profile found 18 statuses and 17 branches; the CRM export has 12 segment spellings:

```
SELECT segment, COUNT(*) AS customers FROM stg_customers_raw GROUP BY 1 ORDER BY 2 DESC;
```

segment	customers
Retail	2521
Hospitality	1398
Wholesale	658
Retail	99
RETAIL	86
retail	85
hospitality	45
HoReCa	38
Hotel/Restaurant	32
wholesale	26
WHOLESALE	21
Distributor	18

(12 rows)

The second **Retail** has a trailing space. **HoReCa** (hotels, restaurants, cafés) and **Hotel/Restaurant** are other names for **Hospitality**; **Distributor** is **Wholesale**.

Two steps: normalize, then map

1. **Normalize** the text so that differences which never matter disappear: `LOWER(TRIM(value))` removes case and outer spaces. In Excel, `=LOWER(TRIM(A2))`; in Power Query, **Transform** → **Format** → **Trim**, then **lowercase**. That alone reduces the segment spellings from 12 to 6 and the statuses from 18 to 7.
2. **Map** the normalized values to the standard ones with a **mapping table** (also called a lookup or crosswalk table): one row per known variant.

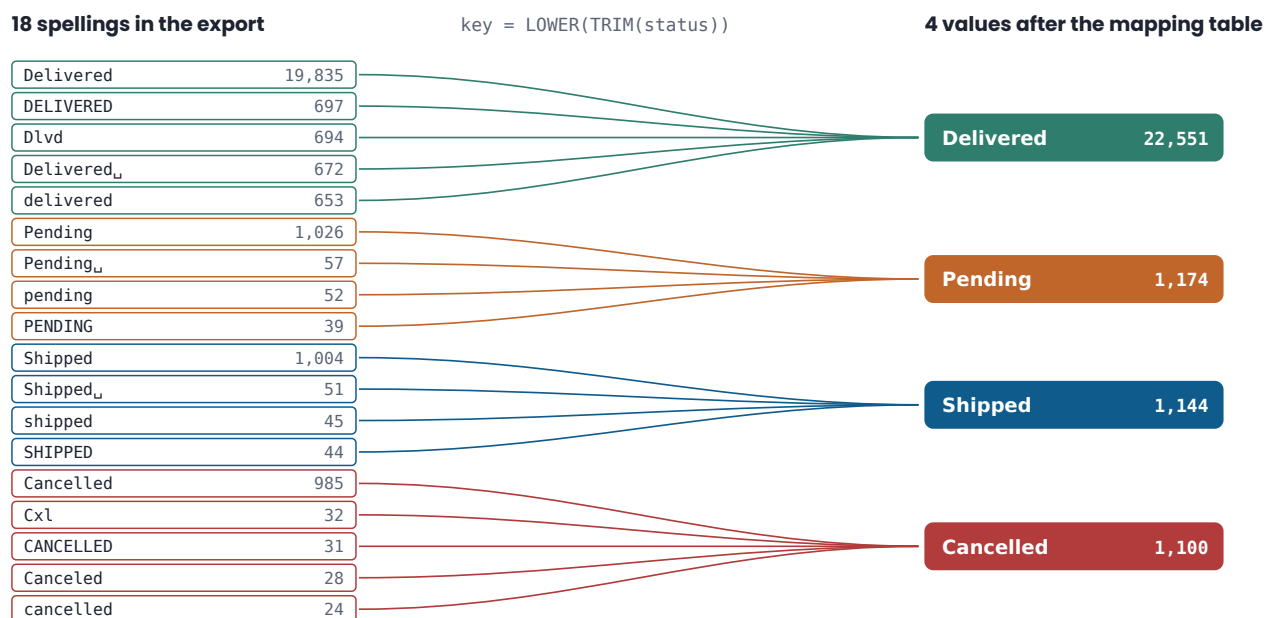
raw_value	clean_value
delivered	Delivered
dlvd	Delivered
cancelled	Cancelled
canceled	Cancelled

raw_value	clean_value
cxl	Cancelled
shipped	Shipped
pending	Pending

The companion scripts create `map_status`, `map_branch`, `map_city`, and `map_segment`. The cleaning query joins to them:

```
SELECT ..., ms.clean_value AS status, mb.clean_value AS branch
FROM typed t
LEFT JOIN map_status ms ON ms.raw_value = LOWER(TRIM(t.status))
LEFT JOIN map_branch mb ON mb.raw_value = LOWER(TRIM(t.branch));
```

Figure 14.4 shows the effect on status.



Counts are raw data rows (duplicates still included). _ marks a trailing space. Unmapped values fail a validation rule instead of disappearing.

Figure 14.4 – Normalizing and mapping turns 18 spellings into 4 values. The counts include the duplicate rows, which are removed separately.

Why a table instead of a long `CASE` expression or nested `IF` s?

- **It's data, not code.** A business user can review and extend it. When a new spelling appears next quarter, someone adds a row; nobody edits a query.
- **It's reusable.** The same `map_city` fixes cities in the CRM export, the order export, and next year's supplier list.
- **It makes unmapped values visible.** With a `LEFT JOIN`, a value that isn't in the table becomes `NULL`, and a validation rule (section 14.9) can count and list them.

Watch out: a CASE with ELSE hides new spellings. `CASE ... WHEN 'dlvd' THEN 'Delivered' ELSE status END` passes any unknown value through unchanged, so a new spelling such as `Deliverd` quietly becomes a fifth status. Map with a `LEFT JOIN` and **fail loudly** when the result is `NULL`.

Old names, abbreviations, and typos

Cities show why a mapping table needs a human to build it. Compare the CRM's city values with Riverstone's reference list of cities (here, the cities in the `customers` table):

```
SELECT city, COUNT(*) AS customers FROM stg_customers_raw
WHERE LOWER(TRIM(city)) NOT IN (SELECT LOWER(clean_value) FROM map_city WHERE clean_value IS NOT
NULL)
AND TRIM(city) NOT IN (SELECT DISTINCT city FROM customers WHERE city IS NOT NULL)
GROUP BY city ORDER BY customers DESC, city;
```

city	customers
New Delhi	25
NULL	24
Poona	24
N/A	22
hyderabad	22
-	20
Madras	20
unknown	19
Gurgaon	17
Trivandrum	17
Mysore	16
Bombay	15
Vizag	15
Bangalore	9
Calcutta	9
(15 rows)	

No function turns **Bombay** into **Mumbai**, **Madras** into **Chennai**, **Poona** into **Pune**, or **Vizag** into **Visakhapatnam**: those are old names and nicknames that only a person who knows India would map. **New Delhi** is a judgment call (it's a district within Delhi); Riverstone's sales regions treat it as Delhi, so the mapping follows the business rule and says so. After mapping, the CRM's 64 distinct city values become 39 real cities.

For spreadsheet users, the same approach is a two-column mapping sheet and a lookup: `=XLOOKUP(LOWER(TRIM(C2)), CityMap[raw_value], CityMap[clean_value], PROPER(TRIM(C2)))`. The fourth argument keeps values that aren't in the map (tidied with `PROPER`), and a separate check lists them. In Power Query, merge with the mapping query on the normalized column (a left outer join) and expand `clean_value`.

Tool note: case sensitivity differs. PostgreSQL compares text case-sensitively, so `'Delivered' = 'DELIVERED'` is false and the profile shows every spelling. MySQL's default collation is case-insensitive, so `GROUP BY status` merges `Delivered`, `DELIVERED`, and `delivered` into one group and hides the problem (Chapter 12, section 12.16). Profile in MySQL with `GROUP BY BINARY status` or a case-sensitive collation (section 14.12).

14.6 Outliers: error or reality?

An **outlier** is a value far from the others. Some outliers are errors; some are the most important rows in the dataset. The job is to find them and then **find out** which kind they are, not to delete everything unusual.

Finding outliers with context

The profile showed a maximum quantity of 700 pieces on a line. Is that possible? Compare with history:

```
SELECT oi.product_id, MAX(oi.quantity) AS max_qty_before_q4 FROM order_items oi JOIN orders o USING
(order_id)
WHERE o.order_date < '2025-10-01' GROUP BY 1 ORDER BY 1;
```

product_id	max_qty_before_q4
101	90
102	75
103	90
104	90
105	65
106	50
107	90
108	90

(8 rows)

In 33 months of order history, no line was ever above 90 pieces. After converting cartons (section 14.7), six Q4 lines exceed that:

```
SELECT c.order_item_id, c.product_id, c.quantity, c.unit_price, c.branch FROM clean_order_lines c
WHERE c.quantity > 90 ORDER BY c.quantity DESC;
```

order_item_id	product_id	quantity	unit_price	branch
187761	101	700	430	Bengaluru
200981	101	450	430	Bengaluru
206224	108	450	290	Mumbai H0
191340	108	400	290	Mumbai H0
205978	105	350	1400	Mumbai H0
209166	101	100	430	Mumbai H0

Every one is an exact multiple of 10, and dividing by 10 gives an ordinary quantity (70, 45, 45, 40, 35, 10). That's the signature of a **typed extra zero**. But "looks like a typo" isn't proof: a hotel chain *could* order 700 storage boxes for a new property. The right action is to **quarantine** the six lines, send the list to the branches, and correct them from the original order documents. At Riverstone's prices, line 187761 alone would add ₹257,355 of revenue if it's wrong and nobody checks (630 extra boxes at ₹430, less the line's 5% discount).

Methods for spotting outliers

Method	How	Good for	Watch out
Business limits	Values outside what's possible or ever seen (history, product rules)	Quantities, prices, discounts, dates	Limits change; review them yearly
Percentiles	Flag values above the 99th percentile, or below the 1st	Any numeric column	Always flags 1% of rows, even in clean data
IQR rule	Below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ (Chapter 21)	Roughly symmetric data	Skewed business data (order values) flags many real big orders
Z-score	More than 3 standard deviations from the mean	Near-normal data	A few huge errors inflate the standard deviation and hide themselves
Within-group comparison	Compare a value with its own group (product, customer, month)	Mixed populations	Small groups have unstable ranges
Visual	Box plot, histogram, scatter (Chapter 15)	Seeing the shape	Doesn't scale to hundreds of columns

For this export, the **business limit** (the historical maximum per product, or overall) is the strongest method: it's based on what Riverstone has actually sold, and anyone can understand it. In a spreadsheet: `=IF(F2>90,"check","")` in a helper column, or conditional formatting (Chapter 10). In Power Query: **Add Column** → **Conditional Column** (`if [quantity] > 90 then "check" else null`). In pandas: `df[df.quantity > 90]` .

Watch out: don't clip or delete outliers in reporting data. "Capping" values at the 99th percentile is a technique for some statistical models. In a sales report it silently changes real transactions. Keep every row, flag the suspects, and decide per row.

Outliers that are real

The profile also shows Wholesale lines of 85 or 90 Industrial Crates, and an October with far higher revenue than July. Those are **reality**: bulk buyers, and Riverstone's festive-season peak (Chapter 11). Removing them would make the data "cleaner" and the analysis wrong. An outlier check produces questions; the business answers them.

14.7 Dates, time zones, units, and currency

Parsing mixed date formats safely

The profile found four date formats. The safe approach is to **parse each known format explicitly**, and leave anything else unparsed (NULL) for review. Never rely on a tool's automatic date guessing: it will happily read `05-11-2025` as 11 May in one system and 5 November in another.

In SQL, the cleaning script splits each recognized format into year, month, and day, then builds the date:

```
CASE WHEN order_date ~ '^\\d{2}[-/]\\d{2}[-/]\\d{4}$' THEN substr(order_date,7,4)::int -- DD-MM-YYYY
or DD/MM/YYYY
      WHEN order_date ~ '^\\d{4}-\\d{2}-\\d{2}$' THEN substr(order_date,1,4)::int END AS y
```

(with matching expressions for month and day), and then:

```
CASE WHEN order_date ~ '^\\d{5}$' THEN DATE '1899-12-30' + order_date::int -- Excel serial
number
      WHEN y IS NOT NULL AND m BETWEEN 1 AND 12
        AND EXTRACT(MONTH FROM make_date(y, m, 1) + (dd - 1)) = m AND dd >= 1
      THEN make_date(y, m, 1) + (dd - 1) END AS parsed_date
```

How it works:

- The regular expression decides the format. `^\\d{2}[-/]\\d{2}[-/]\\d{4}$` means "exactly two digits, a dash or slash, two digits, a dash or slash, four digits". Anything that doesn't match exactly gets no year, and so no date.
- **Excel serial numbers** count days from 30 December 1899, so `45940` is `DATE '1899-12-30' + 45940` = 10 October 2025.
- **Impossible dates** are caught without errors. `make_date(2025, 11, 1) + 30` is 1 December, not 31 November; because its month (12) isn't the month written (11), the test fails and `parsed_date` is NULL. This trick avoids the error PostgreSQL raises for `to_date('31-11-2025', 'DD-MM-YYYY')`, and it works the same way in MySQL (section 14.12).

Nine lines have impossible dates. Each also has an entry timestamp, which gives a reliable repair:

```
SELECT s.order_item_id, s.order_date AS exported_date, c.entered_at_ist, c.order_date AS
repaired_date
FROM clean_order_lines c JOIN (SELECT DISTINCT order_item_id, order_date FROM stg_orders_raw) s ON
s.order_item_id = c.order_item_id::text
WHERE c.date_repaired ORDER BY 1;
```

order_item_id	exported_date	entered_at_ist	repaired_date
192992	31-09-2025	2025-10-28 19:23:00	2025-10-28
193042	31-11-2025	2025-10-28 09:06:00	2025-10-28
193046	32-10-2025	2025-10-28 10:53:00	2025-10-28
193051	32-10-2025	2025-10-28 20:51:00	2025-10-28
201997	31-11-2025	2025-11-28 22:05:00	2025-11-28
202048	31-11-2025	2025-11-28 16:01:00	2025-11-28
202098	32-10-2025	2025-11-30 10:29:00	2025-11-30
209598	31-11-2025	2025-12-28 20:39:00	2025-12-28
209607	31-09-2025	2025-12-28 12:43:00	2025-12-28

(9 rows)

September has 30 days, November has 30, and no month has 32. The entry dates show what the dates should have been, often the 28th, which suggests someone typed a slip of a key. The repair is logged (section 14.10), not hidden.

Watch out: a date that parses isn't necessarily right. 05-11-2025 parsed as day-first is a valid date, and so is the same text parsed month-first. Only knowledge of the source tells you which format a system writes. Check parsed dates against an independent clue: the entry timestamp, the order ID sequence, or the export's date range. Here, every repaired and parsed date falls inside Q4 2025, and a validation rule in section 14.9 confirms it.

In Excel and Power Query. Add a custom column that tries each format in turn, using `try ... otherwise` :

```
= let t = Text.Trim([order_date]) in
  if Text.Length(t) = 5 and Value.Is(Value.FromText(t), type number) then Date.From(Number.From(t))
  else try Date.FromText(t, [Format = "dd-MM-yyyy", Culture = "en-IN"])
  otherwise try Date.FromText(t, [Format = "dd/MM/yyyy", Culture = "en-IN"])
  otherwise try Date.FromText(t, [Format = "yyyy-MM-dd", Culture = "en-IN"])
  otherwise null
```

`Date.FromText` with an explicit `Format` reads exactly that pattern and raises an error for 31-11-2025 , which `try ... otherwise` turns into the next attempt and finally `null` . `Date.From` on a number converts an Excel serial date. In a worksheet,

`=IFERROR(DATE(RIGHT(C2,4),MID(C2,4,2),LEFT(C2,2)),"check")` handles day-first text (Chapter 10) but not impossible dates: Excel's `DATE(2025,11,31)` quietly returns 1 December. Test with

`=AND(DAY(D2)=VALUE(LEFT(C2,2)), MONTH(D2)=VALUE(MID(C2,4,2)))` .

Time zones

The export's `entered_at_utc` column records when each line was entered, in **UTC** (Coordinated Universal Time), as many cloud systems do: 2025-10-01T07:06:00Z , where `Z` means UTC. India Standard Time (IST) is UTC + 5 hours 30 minutes, with no daylight saving time.

The difference matters at the edges of the day. A web-portal order entered at 00:05 IST on 1 November is 18:35 UTC on **31 October**. Group entries by their UTC date, and that order lands in the wrong day, and the wrong month:

```
SELECT COUNT(*) AS lines, COUNT(*) FILTER (WHERE (entered_at_ist - INTERVAL '5 hours 30
minutes')::date <> order_date) AS utc_date_differs FROM clean_order_lines;
```

```
lines | utc_date_differs
-----+-----
25832 | 1598
(1 row)
```

1,598 lines (6.2%) have a different calendar date in UTC than in India. The cleaning script converts to Indian time once, in one column (`entered_at_ist`), using the database's time-zone rules:

```
(entered_at_utc::timestampz AT TIME ZONE 'Asia/Kolkata') AS entered_at_ist
```

`::timestamp`tz reads the text as an absolute moment (the `Z` says UTC), and `AT TIME ZONE 'Asia/Kolkata'` gives the local clock time. MySQL can use `CONVERT_TZ(t, '+00:00', '+05:30')`, or named zones once its time-zone tables are loaded; Power Query uses `DateTimeZone.SwitchZone(DateTimeZone.FromText([entered_at_utc]), 5, 30)`; pandas uses `tz_convert("Asia/Kolkata")`.

Simplification note. Using a fixed +5:30 offset is safe for India. For countries with daylight saving time, always use named time zones (`Europe/London` , `America/New_York`), because the offset changes twice a year.

Units

The Delhi branch enters some quantities in **cartons** of 10 (`qty_unit = 'CTN'`). Mixing units in one column is one of the most damaging problems in business data, because every value looks plausible: 3 cartons looks like a small order of 3 pieces.

The fix is a **conversion to one unit**, driven by the unit column:

```
NULLIF(quantity, '')::int * CASE WHEN qty_unit = 'CTN' THEN 10 ELSE 1 END AS quantity
```

In a real business, the conversion factor belongs in the product master (a crate and a bottle don't come in the same carton size); here every Riverstone carton holds 10 pieces. Spreadsheet:

`=E2*IF(G2="CTN",10,1)` . Power Query: `if [qty_unit] = "CTN" then [quantity] * 10 else [quantity]` .

The same thinking applies to other units you'll meet: kilograms and tonnes, rupees and lakhs, percentages and fractions. Kolkata's spreadsheet upload records discounts as **fractions** (`0.05` for 5%). The rule "a discount above 0 and below 1 is a fraction" is safe here because Riverstone's real discounts are whole percentages; in data where 0.5% discounts exist, you'd need the source system's documentation instead.

Currency and number text

Stripping currency text looks routine and hides a classic trap:

```
SELECT unit_price AS exported, regexp_replace(unit_price, '^[^0-9.]', '', 'g') AS naive_strip,
       regexp_replace(regexp_replace(unit_price, '^[^0-9.]', ''), ',', '', 'g') AS correct
FROM (VALUES ('₹1,400.00'), ('Rs. 430'), ('750')) AS v(unit_price);
```

exported	naive_strip	correct
₹1,400.00	1400.00	1400.00
Rs. 430	.430	430
750	750	750

(3 rows)

The **naive** approach, "keep only digits and the decimal point", turns `Rs. 430` into `.430`, which converts to **0.43**: the full stop in the abbreviation `Rs.` looks like a decimal point. Every `Rs.` price would become a thousandth of its value, and nothing would fail. The **correct** approach removes the

leading non-digit characters (`^[^0-9]+`), then the thousands separators.

A validation rule catches mistakes like this: after conversion, every price must equal the product's list price for 2025 (section 14.9).

Watch out: thousands and decimal separators. `1,400.00` uses the Indian and English convention. Much of Europe writes `1.400,00`. Converting text to numbers with the wrong culture turns one into the other. In Power Query, use **Change Type → Using Locale** or `Number.FromText(x, "en-IN")`; in Excel, **Data → Text to Columns → Advanced** sets the separators; in pandas, `thousands=","` and `decimal="."` in `read_csv`.

14.8 Joining messy sources

Most analysis combines sources: orders with customers, customers with CRM contacts, sales with targets. A join on dirty keys fails quietly: unmatched rows vanish from inner joins, or fill with NULL in left joins, and the totals shrink.

Measure the match rate before you trust a join

```
SELECT COUNT(*) AS lines,
       COUNT(c.customer_id) AS matched_as_exported,
       ROUND(100.0 * COUNT(c.customer_id) / COUNT(*), 1) AS match_pct
FROM (SELECT DISTINCT * FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$') s
LEFT JOIN customers c ON LPAD(c.customer_id::text, 4, '0') = s.customer_code;
```

lines	matched_as_exported	match_pct
25832	25299	97.9

(1 row)

A **match rate** of 97.9% sounds good. It isn't: 533 order lines (2.1%) have no customer, so any report by customer, segment, or city loses them, and they're not random. They're all Kolkata lines whose codes lost their leading zeros (`124` instead of `0124`). Codes above 999 still have four digits and match, which is exactly why the problem is hard to spot: most Kolkata lines look fine.

Repair the key, and measure again:

```
SELECT COUNT(*) AS lines, COUNT(c.customer_id) AS matched_after_lpad
FROM (SELECT DISTINCT * FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+$') s
LEFT JOIN customers c ON LPAD(c.customer_id::text, 4, '0') = LPAD(TRIM(s.customer_code), 4, '0');
```

lines	matched_after_lpad
25832	25832

(1 row)

100%. `LPAD(text, 4, '0')` pads on the left with zeros to four characters; `TRIM` first, so a stray space doesn't count as a character. Spreadsheet: `=TEXT(D2,"0000")` for numbers or `=RIGHT("0000"&TRIM(D2),4)` for text; Power Query: `Text.PadStart(Text.Trim([customer_code]), 4, "0")`; pandas: `.str.strip().str.zfill(4)`.

A checklist for joining messy sources

1. **Profile both keys first:** type, length, pattern, blanks (section 14.2).
2. **Normalize both sides the same way:** trim, case, padding, removing separators (`C-0124` vs `0124`).
3. **Measure the match rate from each side:** what share of orders have a customer, and what share of customers have orders? An anti-join (Chapter 12) lists the unmatched rows.
4. **Check the grain before joining:** if the customer list has 48 duplicate customers, joining orders to it by name would **fan out** (Chapter 12): each of those orders would appear twice.
5. **Look at unmatched rows by group** (branch, month, source). A concentrated gap has a cause.
6. **Reconcile totals before and after the join:** the sum of revenue must not change when you add customer attributes.

Watch out: joining on names. Names are the worst join keys: spelling, case, spaces, and legal suffixes all vary, and two different businesses can share a name. Join on codes. When a source has only names (a supplier's price list, a spreadsheet from a partner), build a match key, match, and keep a reviewed mapping table from their names to your codes, so the matching is done once.

14.9 Validation rules: checks that must return zero

A **validation rule** is a test that every row of clean data must pass. Written as a query or formula that counts failures, each rule should return **0**. Rules turn "I think it's clean" into "these eight statements are true", and they run again, unchanged, on next quarter's file.

Riverstone's rules for clean order lines

```
SELECT * FROM (
  SELECT 'status not mapped' AS rule, COUNT(*) AS failures FROM clean_order_lines WHERE status IS
  NULL
  UNION ALL SELECT 'branch not mapped', COUNT(*) FROM clean_order_lines WHERE branch IS NULL
  UNION ALL SELECT 'date outside Q4 2025', COUNT(*) FROM clean_order_lines WHERE order_date NOT
  BETWEEN '2025-10-01' AND '2025-12-31'
  UNION ALL SELECT 'unknown customer', COUNT(*) FROM clean_order_lines c WHERE NOT EXISTS (SELECT 1
  FROM customers x WHERE LPAD(x.customer_id::text,4,'0') = c.customer_code)
  UNION ALL SELECT 'unknown product', COUNT(*) FROM clean_order_lines c WHERE NOT EXISTS (SELECT 1
  FROM products p WHERE p.product_id = c.product_id)
  UNION ALL SELECT 'price differs from list price', COUNT(*) FROM clean_order_lines c JOIN products
  p USING (product_id) WHERE c.unit_price <> p.unit_price
  UNION ALL SELECT 'discount outside 0-15', COUNT(*) FROM clean_order_lines WHERE discount_pct NOT
  BETWEEN 0 AND 15
  UNION ALL SELECT 'quantity missing or above 90', COUNT(*) FROM clean_order_lines WHERE quantity
  IS NULL OR quantity > 90
) AS checks
ORDER BY failures DESC, rule;
```

rule	failures
quantity missing or above 90	20
branch not mapped	0
date outside Q4 2025	0
discount outside 0-15	0
price differs from list price	0
status not mapped	0
unknown customer	0
unknown product	0
(8 rows)	

(Wrapping the `UNION ALL` in a subquery lets one `ORDER BY` sort all the rules; without it, the rows can come back in any order.) Seven rules pass. The eighth fails on exactly the 20 lines already quarantined: 14 missing quantities and 6 above the historical maximum. A rule that fails on known, logged rows is fine; a rule that fails on rows you can't explain stops the process.

Notice what the rules protect against:

- **status not mapped / branch not mapped:** a new spelling next quarter fails loudly instead of creating a fifth status.
- **price differs from list price:** would have caught the `Rs.` trap from section 14.7 (every such price would be 0.43 instead of 430). It also catches a real business event: a special price or a data-entry error in the ERP.
- **discount outside 0-15:** would have caught fractions converted twice (500%) or not at all (0.05 as a percent is inside the range, which is why the fraction conversion needs its own reasoning, not only a range rule).
- **unknown customer / unknown product:** referential integrity, the rule a database's foreign keys enforce (Chapter 12), checked by hand because an export has no foreign keys.

Types of validation rules

Type	Question	Example
Not null	Is a required value present?	<code>order_id IS NOT NULL</code>
Type and format	Does the value have the right shape?	<code>customer_code</code> is four digits; email matches <code>^[^@\\s]+@[^@\\s]+\\. [^@\\s]+\$</code>
Allowed values	Is it one of the permitted categories?	<code>status</code> in (Delivered, Shipped, Pending, Cancelled)
Range	Is it within possible limits?	<code>discount</code> 0-15; <code>order date</code> not in the future
Uniqueness	Does each key appear once?	<code>order_item_id</code> unique
Referential	Does the code exist in its master list?	<code>customer</code> , <code>product</code> , <code>sales rep</code>
Cross-field	Are related fields consistent?	a Cancelled order has no delivery date; <code>signup_date</code> <= <code>first order date</code>
Reconciliation	Do totals agree with an independent source?	lines and revenue match the ERP (section 14.10)

The customer export shows several of these on one query:

```
SELECT COUNT(*) AS customers, COUNT(*) FILTER (WHERE city IS NULL) AS city_missing, COUNT(*) FILTER
(WHERE email IS NULL) AS email_missing,
      COUNT(*) FILTER (WHERE email_invalid) AS email_invalid, COUNT(*) FILTER (WHERE
signup_in_future) AS signup_in_future
FROM clean_customers;
```

```
customers | city_missing | email_missing | email_invalid | signup_in_future
-----+-----+-----+-----+-----
      5027 |          100 |          150 |           58 |              5
(1 row)
```

The 58 invalid emails (such as `accounts.saffronkitchenware.example.com`, with no `@`) and 5 signup dates in the 2060s (`2060-05-03` and similar) can't be repaired from the export. They go to the CRM owner as a list, exactly like Chapter 1's Western Logistics signing up in 2062.

Validation in Excel and Power Query

- **Data validation** (Chapter 10, section 10.11) prevents bad entries in sheets people type into.
- **A checks sheet**: one row per rule, a `COUNTIFS` or `SUMPRODUCT` formula that counts failures, and conditional formatting that turns any non-zero count red. `=COUNTIFS(Clean[status], "<>Delivered", Clean[status], "<>Shipped", Clean[status], "<>Pending", Clean[status], "<>Cancelled")` counts status failures.
- **Power Query**: a conditional column per rule, or a separate "checks" query that references the clean query, filters the failures, and loads its row count. Loading failures to their own sheet makes them impossible to ignore.

Tool note: validation as a pipeline stage. In data engineering, rules like these run automatically every time data loads, and a failure stops the pipeline or alerts the owner. Tools such as dbt tests and Great Expectations exist for exactly that; Chapter 47 covers them. The rules are the same ones you write here.

14.10 Reconciling and documenting every decision

Reconcile the clean table to the source

Validation proves the data follows the rules. **Reconciliation** proves it matches reality. For this export, reality is Riverstone's ERP, loaded as `riverstone_full`:

```
SELECT COUNT(*) AS lines_in_both,
       SUM(CASE WHEN c.quantity = oi.quantity AND c.unit_price = oi.unit_price AND c.discount_pct =
oi.discount_pct THEN 1 ELSE 0 END) AS lines_identical
FROM clean_order_lines c JOIN order_items oi USING (order_item_id);
```

```
lines_in_both | lines_identical
-----+-----
          25832 |             25812
(1 row)
```

All 25,832 cleaned lines exist in the ERP, and 25,812 are identical in quantity, price, and discount. The other 20 are precisely the quarantined lines (a missing or impossible quantity). The companion check `checks/ch14_compare_clean.py` goes further and compares every column of every line, including dates, codes, statuses, branches, and reps: zero mismatches, for the PostgreSQL, MySQL, and pandas versions alike.

Then reconcile the number that matters:

```
SELECT ROUND(SUM(quantity * unit_price * (1 - discount_pct / 100)), 2) AS net_revenue_clean
FROM clean_order_lines WHERE status <> 'Cancelled'
   AND order_item_id NOT IN (SELECT order_item_id FROM dq_order_issues WHERE action =
'quarantined');
```

```
net_revenue_clean
-----
      423561010.50
(1 row)
```

The ERP's true Q4 2025 net revenue is ₹423,872,808.00. The difference, ₹311,797.50, is exactly the true value of the quarantined non-cancelled lines. **Every rupee of difference is explained.** That sentence, with its numbers, is what a reconciliation should end with.

In real work you rarely have the full truth. You reconcile to what you have: the ERP's own summary screen, finance's month-end total, the row count the source team reports, or last month's approved figures. Two independent totals that agree are strong evidence; a difference you can explain line by

line is nearly as good; a difference you can't explain means the cleaning isn't finished.

The cleaning log

Every repair and every quarantine is a decision someone may question later. Record them in a table:

```
SELECT issue, action, COUNT(*) AS lines FROM dq_order_issues GROUP BY issue, action ORDER BY lines DESC;
```

issue	action	lines
quantity missing	quarantined	14
impossible or unreadable date	repaired from entry time (IST)	9
product_id missing	repaired from unit price	8
quantity above historical maximum	quarantined	6

(4 rows)

`dq_order_issues` has one row per affected line, so anyone can see exactly which lines were touched and why. The log also needs the **rules** that changed many rows at once, which aren't row-level issues:

#	Step	Rule	Rows affected	Decided by	Why
1	Remove non-data rows	<code>order_item_id</code> must be all digits	7 (6 headers, 1 footer)	analyst	export artifacts
2	Remove exact duplicates	identical in every column	137	analyst	export glitch; IDs repeat with identical values
3	Parse dates	4 explicit formats; Excel serials from 1899-12-30	25,832	analyst	branch offices enter dates differently
4	Repair impossible dates	use entry date in IST	9	analyst, confirmed with ERP team	entry date always equals order date
5	Pad customer codes	<code>LPAD(TRIM(code), 4, '0')</code>	533	analyst	Kolkata's upload drops zeros
6	Repair missing product	unique 2025 list price → product	8	analyst	prices are unique per product
7	Convert cartons	$CTN \times 10$	141	Delhi branch head	carton size is 10 for all products
8	Convert fraction discounts	$0 < d < 1 \rightarrow d \times 100$	1,362	analyst	Riverstone discounts are whole percentages
9	Strip currency text	remove leading non-digits, then commas	2,300	analyst	Rs. trap avoided
10	Map status and branch	mapping tables	3,100 and 1,400	sales operations	18 and 17 spellings
11	Quarantine missing quantity	exclude from revenue	14	analyst	can't be recovered
12	Quarantine impossible quantity	above historical maximum of 90	6	analyst; branches to confirm	typed extra zero suspected

#	Step	Rule	Rows affected	Decided by	Why
13	Keep missing sales rep	label "(unassigned)" in reports	812	sales operations	not needed for revenue

(Rows affected are counted after duplicates are removed.)

A log like this does three jobs: it lets a manager approve the decisions, it lets a colleague rerun or challenge them, and it tells the source-system owners what to fix so that next quarter's export is cleaner. Save it next to the query or Power Query file, in version control (Chapter 26).

14.11 The whole pipeline in Excel and Power Query

SQL isn't the only way to build a repeatable cleaning pipeline. Power Query records the same steps, and next quarter's export is one **Refresh** (Chapter 11, section 11.7). Here's the order-export pipeline as applied steps. The companion file `companion/ch14/clean_orders.pq` holds the full M code; paste it into **Data → Get Data → From Other Sources → Blank Query → Advanced Editor** and change the file path.

Queries to create first (each loaded as **Only Create Connection**):

- `StatusMap`, `BranchMap`, `CityMap`: from `status_map.csv` and your completed mappings, with `raw_value` lower-case and trimmed.

The `CleanOrders` query:

1. **Source. From Text/CSV, Do not detect data types, Transform Data.** Delete **Changed Type**.
2. **Remove non-data rows.** Filter `order_item_id` with **Text Filters → Does Not Equal** `order_item_id`, and remove empty values. More robustly, add a custom column `= Text.Length(Text.Select([order_item_id], {"0".."9"})) = Text.Length([order_item_id])` and `[order_item_id] <> ""` and keep `true`. Row count: 25,969.
3. **Remove duplicates.** Select all columns (**Ctrl+A** in the grid), **Home → Remove Rows → Remove Duplicates**. Row count: 25,832.
4. **Trim and normalize text.** Select `customer_code`, `status`, `sales_rep`, `branch`: **Transform → Format → Trim**. Duplicate `status` and `branch` and apply **Format → lowercase** to the copies (these are the join keys).
5. **Map categories.** **Home → Merge Queries:** the lower-case status copy to `StatusMap[raw_value]`, **Left Outer**; expand `clean_value` as `status_clean`. Repeat for branch.
6. **Pad codes.** **Add Column → Custom Column** `= Text.PadStart([customer_code], 4, "0")`.
7. **Parse dates.** Custom column with the `try ... otherwise` expression from section 14.7. Then **Add Column → Custom Column** `entered_at_ist = DateTimeZone.RemoveZone(DateTimeZone.SwitchZone(DateTimeZone.FromText([entered_at_utc]), 5, 30))`, and `order_date_final = if [order_date_parsed] = null then Date.From([entered_at_ist]) else [order_date_parsed]`.

8. **Convert numbers.** `unit_price_num = Number.FromText(Text.Remove(Text.TrimStart([unit_price], {"R","S",".", " ","₹"}), ","), "en-IN"); discount = let d = Number.FromText([discount_pct], "en-IN") in if d > 0 and d < 1 then d * 100 else d; quantity_pcs = if [quantity] = "" then null else Number.FromText([quantity]) * (if [qty_unit] = "CTN" then 10 else 1) .`
9. **Repair product.** `product = if [product_id] = "" then Record.FieldOrDefault([#"430" = 101, #"750" = 102, #"115" = 103, #"620" = 104, #"1400" = 105, #"1150" = 106, #"380" = 107, #"290" = 108], Text.From([unit_price_num]), null) else Number.FromText([product_id]) .`
10. **Flag issues.** Conditional column `issue`: `quantity null` → "quantity missing"; `quantity > 90` → "quantity above historical maximum"; `parsed date null` → "date repaired"; `product was blank` → "product repaired".
11. **Set types and choose columns.** Keep the clean columns, set each type deliberately, and rename.
12. **Load.** `CleanOrders` to a table (or the Data Model), and a second query `Issues` that references `CleanOrders`, filters `issue <> null`, and loads to its own sheet.

Then add a **Checks** sheet: the row count of `CleanOrders` (25,832), the count of unmapped statuses and branches (`=COUNTIFS(CleanOrders[status_clean], "")`, 0), the count of `Issues` (37), and the non-cancelled revenue excluding quarantined lines (₹423,561,010.50).

Tool note: Text.TrimStart and the Rs. trap. `Text.TrimStart(text, {"R","s",".", " ","₹"})` removes only those characters, only from the start, so `Rs. 430` becomes `430` and `₹1,400.00` becomes `1,400.00`. `Text.Select(text, {"0".."9", "."})` would reproduce the section 14.7 trap and turn `Rs. 430` into `.430`.

Google Sheets. Sheets has no Power Query. For a dataset this size, clean in SQL or Python and bring the result into Sheets; for smaller files, use helper columns with the same logic: `=LPAD -style`
`=RIGHT("0000"&TRIM(D2),4)`, `=VLOOKUP(LOWER(TRIM(J2)), StatusMap!A:B, 2, FALSE)`,
`=REGEXREPLACE(H2, "^[^0-9]+", "")` then `=VALUE(SUBSTITUTE(..., ",", ""))`, and
`=IFERROR(DATE(...))` with the checks from section 14.7. `QUERY` (Chapter 11, section 11.10) then summarizes the cleaned range.

14.12 Running this chapter in MySQL

The companion scripts `ch14_load_mysql.sql` and `ch14_clean_mysql.sql` build the same tables in MySQL 8.0, and the cleaned result is identical line for line. The differences are worth knowing.

Loading. `LOAD DATA LOCAL INFILE` needs local loading enabled on both sides (`SET GLOBAL local_infile = 1`; on the server, `--local-infile=1` on the client), `LINES TERMINATED BY '\r\n'` for Windows line endings, and `CHARACTER SET utf8mb4` for ₹. Empty CSV fields load as empty strings, not NULL, so the cleaning uses `NULLIF(col, '')` everywhere.

Profiling patterns. `REGEXP_REPLACE` replaces every match by default, without a `'g'` flag:

```
SELECT REGEXP_REPLACE(order_date, '[0-9]', '9') AS date_pattern, COUNT(*) AS `rows`
FROM stg_orders_raw WHERE order_item_id REGEXP '^[0-9]+$' GROUP BY 1 ORDER BY `rows` DESC;
```

date_pattern	rows
99-99-9999	20890
9999-99-99	3227
99/99/9999	1812
99999	40

(`rows` is a reserved word in MySQL 8, so it needs backticks; Chapter 13, section 13.8.)

The case-insensitive trap. The same status profile as section 14.2 gives a different answer:

```
SELECT status, COUNT(*) AS `rows` FROM stg_orders_raw WHERE order_item_id REGEXP '^[0-9]+$' GROUP
BY status ORDER BY `rows` DESC LIMIT 6;
```

status	rows
Delivered	21185
Pending	1117
Shipped	1093
Cancelled	1040
Dlvd	694
Delivered	672

MySQL's default collation (`utf8mb4_0900_ai_ci`, where `ci` means case-insensitive) groups `Delivered`, `DELIVERED`, and `delivered` together, so the profile looks cleaner than the data is. Trailing spaces still count (that collation doesn't pad), which is why `Delivered` appears separately. To see every spelling, group by `BINARY status` or `status COLLATE utf8mb4_bin`. And remember that it cuts both ways: `WHERE status = 'delivered'` matches all three spellings in MySQL but only one in PostgreSQL.

Match rate. The same query, with `LPAD` accepting a number directly:

```
SELECT COUNT(*) AS `lines`, COUNT(c.customer_id) AS matched_as_exported
FROM (SELECT DISTINCT * FROM stg_orders_raw WHERE order_item_id REGEXP '^[0-9]+$') s
LEFT JOIN customers c ON LPAD(c.customer_id, 4, '0') = s.customer_code;
```

lines	matched_as_exported
25832	25299

Dates. MySQL has no `make_date`; the cleaning script builds the date with `MAKEDATE(y, 1) + INTERVAL (m - 1) MONTH + INTERVAL (dd - 1) DAY` and applies the same "does the month still match?" test. Avoid `STR_TO_DATE('31-11-2025', '%d-%m-%Y')` inside `CREATE TABLE ... AS SELECT`: in strict mode an invalid date raises an error for the whole statement instead of returning `NULL`.

Time zones. `STR_TO_DATE(entered_at_utc, '%Y-%m-%dT%H:%i:%sZ') + INTERVAL 330 MINUTE` (India has no daylight saving time), or `CONVERT_TZ(t, '+00:00', '+05:30')`. Named zones such as `'Asia/Kolkata'` work in `CONVERT_TZ` only after the time-zone tables are loaded

```
(mysql_tzinfo_to_sql).
```

Result.

```
SELECT ROUND(SUM(quantity * unit_price * (1 - discount_pct / 100)), 2) AS net_revenue_clean
FROM clean_order_lines WHERE status <> 'Cancelled'
AND order_item_id NOT IN (SELECT order_item_id FROM dq_order_issues WHERE action =
'quarantined');
```

```
+-----+
| net_revenue_clean |
+-----+
|      423561010.50 |
+-----+
```

The same ₹423,561,010.50 as PostgreSQL.

Task	PostgreSQL	MySQL 8.0
Regex match	<code>col ~ 'pattern'</code>	<code>col REGEXP 'pattern'</code>
Replace all matches	<code>regexp_replace(col, p, r, 'g')</code>	<code>REGEXP_REPLACE(col, p, r)</code>
Conditional count	<code>COUNT(*) FILTER (WHERE ...)</code>	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code>
Distinct whole rows	<code>COUNT(DISTINCT t.*)</code>	<code>COUNT(*) on SELECT DISTINCT * in a subquery</code>
Build a date	<code>make_date(y, m, d)</code>	<code>MAKEDATE(y, 1) + INTERVAL (m-1) MONTH + INTERVAL (d-1) DAY</code>
Time zone	<code>ts::timestampz AT TIME ZONE 'Asia/Kolkata'</code>	<code>CONVERT_TZ(ts, '+00:00', '+05:30')</code>
Case-sensitive grouping	default	<code>GROUP BY BINARY col</code>
Empty CSV field	NULL (unquoted)	empty string
Join by similarity	<code>pg_trgm: similarity(a, b)</code>	no built-in; SOUNDEX is crude

14.13 A first look at cleaning in pandas

Python is taught in its own block: Chapter 17 covers the language from zero and Chapter 18 covers **pandas**, the library for working with tables in Python. If you're reading in file order and haven't reached them yet, treat this section as a preview and come back to it afterwards. This preview shows that the method (load as text, profile, fix, validate, reconcile) is the same in Python, and that a script is as repeatable as a query. The code runs from `companion/ch14; clean_orders_pandas.py` holds the complete pipeline.

```
import pandas as pd
raw = pd.read_csv("orders_q4_2025_export.csv", dtype=str, keep_default_na=False)
print(raw.shape)
```

```
(25976, 13)
```

`dtype=str` loads every column as text and `keep_default_na=False` keeps empty fields as empty strings instead of guessing: the staging-table idea in one line. The file has 25,976 rows and 13 columns.

```
data = raw[raw["order_item_id"].str.fullmatch(r"\d+")]
print(len(data), "data rows;", data.duplicated().sum(), "exact duplicates")
data = data.drop_duplicates()
print(data["order_date"].str.replace(r"\d", "9", regex=True).value_counts())
```

```
25969 data rows; 137 exact duplicates
order_date
99-99-9999    20782
9999-99-99     3210
99/99/9999     1800
99999          40
Name: count, dtype: int64
```

The same seven non-data rows and 137 duplicates as SQL. The pattern counts are smaller than section 14.2's because the duplicates are already gone.

```
print(data["status"].str.strip().str.lower().value_counts())
```

```
status
delivered    21744
pending       1166
shipped       1138
cancelled     1034
dlvd          690
cxl           32
canceled      28
Name: count, dtype: int64
```

Normalizing with `.str.strip().str.lower()` leaves the seven spellings a mapping table must handle.

```
price = pd.to_numeric(data["unit_price"].str.replace(r"^[^0-9]+", "", regex=True).str.replace(", ", ""))
print(price.value_counts().sort_index())
```

```
unit_price
115.0      3606
290.0      5082
380.0      3583
430.0      4985
620.0      3575
750.0      3621
1400.0     1380
Name: count, dtype: int64
```

Every converted price is one of Riverstone's seven list prices for the products sold in Q4 (no garden chairs sell in winter), which is a validation rule passed.


```
print(data["customer_code"].str.len().value_counts().sort_index())
print(data["customer_code"].str.zfill(4).str.len().value_counts())
```

```
customer_code
2         40
3        493
4       25299
Name: count, dtype: int64
customer_code
4        25832
Name: count, dtype: int64
```

`zfill(4)` pads with zeros on the left, like `LPAD`. After padding, every code has four characters.

Run the whole script with `python3 clean_orders_pandas.py`; it prints the same 37 logged issues and ₹423,561,010.50 as SQL, and writes `clean_order_lines_pandas.csv` and `dq_order_issues_pandas.csv`.

Tool note. Tested with Python 3.12 and pandas 3.0.2. pandas 2.x gives the same results; in versions before 2.0, `value_counts()` prints its header differently.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Cleaning in the source file or table	Can't explain or undo a change; next export needs the same edits by hand	Clean into a new table, query, or copy; keep the raw data
Letting tools guess types on load	Codes lose zeros; dates flip; values become errors	Load as text; convert each column deliberately
Profiling only the first rows	Problems in one branch or month go unnoticed	Profile the entire dataset; say what the profile covered
Trusting a file's footer or summary count	Row counts that include headers and duplicates	Count data rows yourself
Treating "N/A", "-", "NULL" text as values	"N/A" appears as a city in reports	Standardize every placeholder to NULL
Filling missing numbers with 0 or an average	Totals look complete and are wrong	Quarantine and report the gap, or keep NULL
Removing duplicates on too few columns	Real repeat purchases disappear	Deduplicate on the key; check that copies are identical
Removing duplicates by ID when values differ	The wrong (older) version survives	Compare distinct rows with distinct IDs; keep the latest version by timestamp
Auto-merging fuzzy matches	Two different businesses combined	Match keys and similarity find candidates; a person confirms
CASE ... ELSE value for mapping	New spellings pass through silently	Mapping table with <code>LEFT JOIN</code> ; fail when unmapped

Mistake	Symptom	Fix
Deleting or capping outliers	Real big orders vanish; errors hide	Flag with business limits; investigate each
Letting a tool guess day-first vs month-first	Dates shift by months with no error	Parse each known format explicitly
Ignoring impossible dates	<code>DATE(2025,11,31)</code> becomes 1 December silently	Test that day and month survive the conversion
Grouping UTC timestamps by date	Late-night entries land on the wrong day or month	Convert to local time once, then group
Mixed units in one column	3 cartons counted as 3 pieces	Convert with the unit column; validate ranges
Stripping "everything except digits and ." from prices	Rs . 430 becomes 0.43	Remove leading non-digits, then separators; validate against list prices
Joining on names or unpadded codes	Match rate below 100%; totals shrink	Normalize keys; measure match rates from both sides
Profiling categories in MySQL with default collation	Case variants hidden	<code>GROUP BY BINARY col</code>
Declaring data clean without reconciliation	A tidy table with the wrong total	Reconcile to an independent total; explain every difference
No cleaning log	Nobody can approve, repeat, or challenge the work	Log every rule, count, and decision

In the real world: the branch league table

On 6 January 2026, Vikram Singh opened the quarterly sales review with a slide the branch heads had already seen by email: Q4 2025 net revenue by branch.

Rank	Branch	Q4 net revenue (₹)
1	Mumbai HO	132,385,776
2	Delhi	101,831,086
3	Bengaluru	101,788,506
4	Kolkata	54,539,528

Sandeep Gill, the Regional Sales Manager for the North, had already thanked his Delhi team for "overtaking Bengaluru". Arjun Nair, who ran the South, didn't believe it: his team's order book had been the strongest in the company since September, and they were behind Delhi by ₹42,580, less than one order.

The slide had been built the quick way. Someone had opened `orders_q4_2025_export.csv`, calculated `quantity × price × (1 - discount ÷ 100)`, excluded rows whose status was exactly `Cancelled`, and pivoted by branch. Anita Rao asked Meera Iyer to check it before anyone was congratulated.

Meera started by profiling the export, not by rebuilding the pivot.

1. Which rows were counted? Every row with a price written as Rs. 430 or ₹1,400.00 had been treated as text and left out of the calculation: 2,316 rows, all from Mumbai HO and Bengaluru, where staff paste prices copied from quotations, currency symbols included. That alone explained why both looked small.

2. Were rows counted twice? 137 lines appeared twice in the export, and the quick pivot had included both copies.

3. Were the Delhi numbers in the right units? 141 Delhi lines were in cartons of 10. The pivot had counted 3 cartons as 3 pieces, which understated Delhi, but not by enough to matter.

4. Were the Kolkata discounts right? Kolkata's 1,362 discounted lines stored 5% as 0.05, so the formula applied a discount of 0.05%. Kolkata's revenue was overstated.

5. What about "Cancelled"? 115 cancelled lines were spelled CANCELLED, Canceled, cancelled, or Cxl and had been counted as sales.

She ran the cleaning pipeline from this chapter, logged 37 issues (quarantining 20 lines whose quantities were missing or impossible), and reconciled the result to the ERP: every line matched, and the difference in revenue was exactly the value of the quarantined lines. The corrected table:

Rank	Branch	Quick slide (₹)	Clean (₹)	Change
1	Mumbai HO	132,385,776	150,854,187	+18,468,411
2	Bengaluru	101,788,506	117,619,062	+15,830,556
3	Delhi	101,831,086	102,966,828	+1,135,742
4	Kolkata	54,539,528	52,120,934	-2,418,594
	Total	390,544,896	423,561,011	+33,016,115

(Clean figures exclude cancelled orders and the 20 quarantined lines, rounded to the rupee.)

Bengaluru was second by ₹14.7 million. The company's Q4 revenue was ₹33 million (8.5%) higher than the slide said, and Kolkata, the one branch that had looked better on the slide, was ₹2.4 million lower.

Her note to Anita and Vikram was short:

The Q4 branch slide understated revenue by ₹33.0 million and put Delhi ahead of Bengaluru by mistake. The export mixes currency text, cartons, fraction discounts, duplicate lines, and five spellings of Cancelled; a quick pivot can't handle that. Corrected ranking: Mumbai HO, Bengaluru, Delhi, Kolkata. The clean table reconciles to the ERP line by line. Twenty lines with missing or impossible quantities are excluded until the branches confirm them; the list is attached.

Then she did what made the next review different. She sent the cleaning log to the ERP team with three requests: one date format for every branch, prices as plain numbers, and discounts as percentages everywhere. She put the SQL pipeline and the mapping tables in the team's repository, with the validation rules as the last step. And she added one line to the quarterly review template: "Figures from the cleaned order table; checks passed; issues logged."

What made the difference:

- She **profiled before calculating**, so she found the problems instead of arguing about the result.
 - She looked for errors in **both directions**: most made branches look smaller, one made Kolkata look bigger.
 - She **reconciled to the source** and explained every rupee of the remaining gap.
 - She **fixed the process**: the log went to the people who could stop the mess at its source.
-

Tools

- **PostgreSQL 16** and **MySQL 8.0** (tested on 16.15 and 8.0.46), with DBeaver or MySQL Workbench. The `pg_trgm` extension ships with PostgreSQL (`CREATE EXTENSION pg_trgm;`).
 - **Excel for Windows** (Microsoft 365) with Power Query; Power Query on Mac has fewer features. **Google Sheets** for smaller files.
 - **Python 3.12** with **pandas 3.0.2** (pandas 2.x works) for section 14.13.
 - **Companion files**:
 - `companion/full/` : the full Riverstone dataset (CSV, Parquet, PostgreSQL and MySQL setup scripts) and `DATA_SPEC.md`.
 - `companion/ch14/orders_q4_2025_export.csv` and `customers_crm_export.csv` : the messy exports.
 - `companion/ch14/sql/` : `ch14_load_postgresql.sql`, `ch14_clean_postgresql.sql`, `ch14_load_mysql.sql`, `ch14_clean_mysql.sql`.
 - `companion/ch14/clean_orders.pq` : the Power Query pipeline. `clean_orders_pandas.py` : the pandas pipeline.
 - `companion/ch14/city_map.csv`, `status_map.csv` : starter mapping tables (incomplete on purpose).
 - `companion/ch14/answer_key.json` and `clean_truth_orders_q4_2025.csv` : every planted problem and the correct clean table, for checking your work.
 - `companion/ch14/build_ch14_files.py` : rebuilds the exports from the full dataset (seed 20251014).
 - `checks/ch14_compare_clean.py` : compares your cleaned table with the truth, column by column.
-

The project: clean Riverstone's operations export and write a data-quality report

Goal: turn a messy export into a trusted, repeatable clean table, and write a one-page report that a manager can act on.

Option A: your own data. Use an export you receive regularly. Remove or mask personal and confidential data first. You won't have a truth file, so reconcile to the best independent total you can find (the system's own summary screen, finance, or last period's approved report).

Option B: Riverstone's data. Use `orders_q4_2025_export.csv` and `customers_crm_export.csv`.

Steps

1. **Load as text** in the tool of your choice: SQL staging tables, Power Query with no type detection, or pandas with `dtype=str`.
2. **Profile both files** with the section 14.2 checklist. Write your findings as a table before fixing anything. Don't open the answer key yet.
3. **Build the cleaning pipeline** in steps: non-data rows, duplicates, types, categories (with mapping tables), keys, units, dates and time zones, repairs, and quarantine. Every step is code or recorded steps, never manual edits.
4. **Write at least eight validation rules** that return failure counts, and run them.
5. **Reconcile.** For the order export, compare with `riverstone_full` (or `clean_truth_orders_q4_2025.csv`) using `checks/ch14_compare_clean.py`. Explain every remaining difference.
6. **Keep a cleaning log** with the columns from section 14.10.
7. **Write the data-quality report (one page):**
 - What was received (files, rows, period) and what one clean row represents.
 - The six dimensions from Chapter 1 (accuracy, completeness, consistency, validity, uniqueness, timeliness): a line each with the main finding and a count.
 - The top five problems by business impact, with the rupee effect where you can measure it (the story above shows how).
 - What was fixed, what was quarantined, and what needs a decision from someone else.
 - Three requests to the source-system owners that would prevent the problems.
8. **Compare with the answer key.** Which problems did you find, which did you miss, and which did you "fix" in a way the truth shows was wrong?

What good looks like (Option B): 25,832 clean order lines; 7 non-data rows and 137 duplicates removed; 9 dates and 8 products repaired; 20 lines quarantined; every column matches the truth for the other lines; non-cancelled revenue excluding quarantined lines ₹423,561,010.50, with ₹311,797.50 explained. Customers: 5,027 records, 48 duplicate groups (4,979 businesses), 39 cities, 100 missing cities, 58 invalid and 150 missing emails, 5 impossible signup dates.

Stretch goals

- Make the pipeline run on a second quarter: change the planted-damage seed in `build_ch14_files.py`, rebuild, and rerun without editing your cleaning code. Whatever breaks is a rule that was too specific.
- Add a `pg_trgm` similarity check that lists customer pairs in the same city with similarity above 0.6 that your match key didn't group, and review ten of them.
- Build the same pipeline in a second tool and prove both outputs are identical.

Timed challenge: the CRM clean-up sprint (40 minutes)

In the style of Chapter 11's competition cases: one file, a clock, and answers that can be checked. Use `customers_crm_export.csv` and any tool. No walkthrough; the answers are at the end of the chapter.

Rules of the data: a customer record is one row. Two records are duplicates if their names are the same after trimming, collapsing spaces, ignoring case, and removing a trailing "Pvt Ltd" (with or without dots) or "Private Limited". Missing values include blanks and the texts `N/A`, `-`, `unknown`, and `NULL`. City old names map to current names (Bombay → Mumbai, Bangalore → Bengaluru, Madras → Chennai, Calcutta → Kolkata, Poona → Pune, Gurgaon → Gurugram, Trivandrum → Thiruvananthapuram, Vizag → Visakhapatnam, Mysore → Mysuru, New Delhi → Delhi). Segments: HoReCa and Hotel/Restaurant are Hospitality; Distributor is Wholesale.

- **Level 1:** How many records have a city exactly equal to `Mumbai` as exported? How many records are in Mumbai after cleaning?
- **Level 2:** How many distinct city values are in the export (not counting real blanks), and how many distinct cities after cleaning?
- **Level 3:** How many records are in each segment after cleaning?
- **Level 4:** How many records have no usable city? How many have a missing email, and how many an email without `@`?
- **Level 5:** How many signup dates are written `DD/MM/YYYY`? How many signup dates are after 2025, and what is the earliest of them?
- **Level 6:** How many duplicate groups are there, and how many distinct businesses?
- **Level 7:** Keeping the record with the lowest customer code in each duplicate group, how many businesses are in each segment, and how many in Mumbai?
- **Bonus:** How many Hospitality records are in Bengaluru after cleaning?

You've got it when...

- ☐ You load unfamiliar data as text and convert each column deliberately.
- ☐ You can profile a new file in twenty minutes: rows and grain, key uniqueness, non-data rows, blanks and placeholders, distinct values, patterns, ranges, and relationships, over the whole file.
- ☐ You recognize every form of missing and choose between standardize, repair, quarantine, and keep, with a reason for each.
- ☐ You find exact duplicates on the key, check that copies are identical, and find fuzzy duplicates with match keys and similarity, without auto-merging.
- ☐ You standardize categories with a normalize-then-map approach and a mapping table that fails loudly on new values.
- ☐ You investigate outliers with business limits and history, and never delete or cap real transactions.

- ☐ You parse mixed date formats explicitly, catch impossible dates, convert UTC to local time before grouping, and convert units and currency text safely.
 - ☐ You measure join match rates from both sides before trusting a join.
 - ☐ You write validation rules that return zero, and reconcile the clean table to an independent total until every difference is explained.
 - ☐ You keep a cleaning log that someone else can approve, repeat, and challenge.
 - ☐ You can do the core steps in at least two of Power Query, SQL (PostgreSQL or MySQL), and pandas.
-

Recap

- **Cleaning is a workflow:** load as text, profile, fix, validate, reconcile, document. Never clean the source; make every step repeatable.
 - **Profile first**, over the whole file: rows and grain, key uniqueness, non-data rows, blanks, distinct values, patterns (digits → 9), ranges, and relationships. Riverstone's Q4 export had 7 non-data rows, 137 duplicates, 4 date formats, 18 status spellings, and 6 price formats.
 - **Missing values** come in many spellings (blank, NULL, N/A, -, unknown, the text NULL). Standardize them, then repair from reliable columns, quarantine what the result needs, or keep and label what it doesn't. Never fill with 0 or an average in reporting data.
 - **Duplicates:** deduplicate on the key after checking copies are identical. Fuzzy duplicates need a **match key** (case, spaces, legal suffixes) or similarity; matching choices change results (48 groups by name, 46 by name and city). People confirm merges.
 - **Categories:** normalize (LOWER(TRIM())), then map with a table joined by LEFT JOIN, and fail on unmapped values. Old names like Bombay need human knowledge.
 - **Outliers** are questions. Business limits from history (no line above 90 pieces in 33 months) found six typed extra zeros; quarantine, don't delete.
 - **Dates, time zones, units, currency:** parse each known format explicitly; test that day and month survive; convert UTC to IST before grouping (1,598 lines change date); convert cartons with the unit column; strip leading currency text, not every non-digit (Rs. 430 → 0.43).
 - **Joins:** normalize keys and measure match rates. 97.9% hid 533 Kolkata lines with lost zeros.
 - **Validate and reconcile:** rules that return zero, then a reconciliation that explains every rupee (₹423,561,010.50 clean, ₹311,797.50 quarantined, matching the ERP).
 - **Document** every rule, count, and decision in a cleaning log, and send it to the people who can fix the source.
 - **MySQL** hides case variants in GROUP BY and loads empty CSV fields as empty strings; PostgreSQL loads them as NULL and compares case-sensitively.
-

Practice exercises

Unless an exercise says otherwise, use the staging tables and cleaned tables in `riverstone_full` (PostgreSQL or MySQL), Power Query, or pandas, and the files in `companion/ch14/`.

Warm-up

1. For each problem, name the data-quality dimension from Chapter 1 (accuracy, completeness, consistency, validity, uniqueness, timeliness): (a) a signup date of 2064-02-25; (b) `Bombay` and `Mumbai`; (c) an email without `@`; (d) the same order line exported twice; (e) a customer with no city; (f) Q4 figures used in March to set this month's stock levels.
2. Give two things that go wrong if you open `orders_q4_2025_export.csv` by double-clicking it in Excel instead of loading it as text.
3. Write the digit pattern (every digit replaced by 9) for `05/11/2025`, `Rs. 1,400`, `0124`, and `45940`. What would you suspect each is?
4. A quantity column contains a blank, `0`, and the text `NULL`. What does each probably mean, and what should each become?
5. For each missing value, choose standardize, repair, quarantine, or keep and label, and say why: (a) `product_id` blank on a line with unit price ₹620; (b) `quantity` blank; (c) `sales_rep` blank; (d) city `unknown`.
6. In MySQL, `SELECT status, COUNT(*) FROM stg_orders_raw GROUP BY status` shows fewer spellings than in PostgreSQL. Why, and how do you see them all?

Core

7. Profile the `branch` column. How many spellings are there, and how many remain after `LOWER(TRIM())`? Write the mapping table.
8. Prove that the duplicate rows in the order export are identical copies, and count them.
9. Write the query (or Power Query steps) that lists order lines whose dates can't be parsed. How many are there, and how were they repaired?
10. How many dates in the export are Excel serial numbers? Convert `45940` in SQL, Excel, and Power Query.
11. Measure the customer-code match rate against `customers` before and after padding. Which branch's lines fail to match as exported, and why do only 533 of its lines fail?
12. What does the naive price conversion (keep digits and `.`) return for `Rs. 750` and `₹750.00`? Write a conversion that handles both, and a validation rule that would catch the naive version.
13. How many lines have discounts written as fractions, and from which branch? What rule converts them, and when would that rule be unsafe?
14. How many lines are in cartons? What are their total pieces as written and after conversion?
15. List the lines with quantities above the historical maximum. What would they add to Q4 net revenue as written, compared with the corrected quantities they probably should have?
16. How many order lines have a different calendar date in UTC than in India? How many land in a different **month**?

17. Write a validation rule that every sales rep name in the clean table exists in `employees`. How many failures? How many raw rows had a trailing space in `sales_rep`?
18. Reconcile `clean_order_lines` with `order_items`: how many lines exist in both, and how many are identical in quantity, price, and discount? Explain the difference.
19. Clean the customer export's `segment` column. How many records are in each segment?
20. Count duplicate customer groups with the match key on name only, and on name plus city. Explain the difference in numbers and what it means for choosing a rule.
21. List every form of "missing" in the customer export's `city` column with its count.
22. In Power Query, profile `status` using the top 1,000 rows and then the entire dataset. How many distinct values does each show?
23. In pandas, normalize `status` with `.str.strip().str.lower()`. How many distinct values remain?
24. Rebuild the branch league table from the story: first the quick way (numeric prices only, rows marked exactly `Cancelled` excluded, duplicates and units as written), then clean. Which two branches swap places?

Stretch

25. The product repair in section 14.3 relies on every product having a different price. Write a query that proves it for 2025, and explain what you'd do in 2026 if two products shared a price.
26. Using `pg_trgm`, what is the similarity between `Balaji Distributors` and `BALAJI DISTRIBUTORS`, `Balaji Distrubutors`, and `Balaji Traders`? Where would you set a review threshold, and why?
27. List the customer records with signup dates in the future, and write the message you'd send to the CRM owner.

Think about it

28. A manager asks you to "fill the 14 blank quantities with the average quantity so the report is complete". What do you reply?
29. A colleague suggests fixing the duplicate customers directly in the CRM with an `UPDATE` and `DELETE`, since your match key found them all. What's wrong with that plan, and what should happen instead?
30. You're asked to define "duplicate" for a list of 40,000 sales leads collected from a website form, trade fairs, and purchased lists. Which columns would you use, what normalization would you apply, and what error would you rather make: missing some duplicates or merging some different people?

Key terms

data cleaning · data preparation · staging table · profiling · grain · non-data row · pattern profile · placeholder value · missing value · missing completely at random · missing at random · missing not at random · imputation · standardize · repair · quarantine · fill down · exact duplicate · fuzzy duplicate · match key · similarity · trigram · Jaccard similarity · fuzzy merge · category · normalize · mapping table

(crosswalk) · outlier · business limit · IQR rule · z-score · Excel serial date · impossible date · UTC · IST · time zone · unit conversion · fraction vs percentage · currency text · thousands separator · locale · join key · match rate · anti-join · fan-out · validation rule · referential integrity · cross-field rule · reconciliation · cleaning log · data-quality report · collation

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 15, Data Visualization Principles:** box plots and histograms for spotting outliers, and charts that show data-quality gaps clearly.
- **Chapter 16, Business Intelligence with Power BI:** the same Power Query steps feeding a data model, with scheduled refresh.
- **Chapter 18, Python for Analysts:** pandas properly: reading every format, cleaning with vectorized operations, `merge`, and fuzzy matching libraries.
- **Chapter 20, Automating Reports & Delivering Insights:** running the cleaning and validation rules as the first step of an automated report, and alerting when a rule fails.
- **Chapter 21, Descriptive Statistics & Probability:** percentiles, the IQR, and z-scores behind the outlier methods in section 14.6.
- **Chapter 47:** data-quality testing in pipelines (dbt tests, Great Expectations), where this chapter's rules run on every load.
- **Interview preparation:** the SQL Question Bank (Chapter 71) and the Business Analyst bank (Chapter 76) include "here's a messy dataset; walk me through what you'd check" and deduplication with window functions.

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Accuracy (and validity: a date after today breaks a range rule). (b) Consistency. (c) Validity. (d) Uniqueness. (e) Completeness. (f) Timeliness: the data may be correct but is too old for the decision.
2. Any two of: customer codes such as `0124` lose their leading zeros; day-first dates such as `05-11-2025` may be read month-first or left as text depending on regional settings (Chapter 10, section 10.4); `Rs. 430` and `₹1,400.00` stay as text and are skipped by `SUM`; the repeated header rows become text in number columns, turning whole columns into text; long IDs could be converted to numbers.
3. `05/11/2025` → `99/99/9999`: a date with slashes (day-first or month-first is unknown from the pattern alone). `Rs. 1,400` → `Rs. 9,999`: a price stored as currency text. `0124` → `9999`: a four-digit code (text, keep the zero). `45940` → `99999`: an Excel serial date (10 October 2025).

4. Blank: unknown (not recorded). 0 : probably a real zero, unless the source uses 0 for unknown, which you'd confirm with the owner. The text NULL : a system printed its missing value as a word. Blank and NULL should both become a real NULL; 0 stays 0 unless the source documentation says otherwise. For quantity, a real zero on an order line is itself suspicious and worth a validation rule.
5. (a) **Repair**: ₹620 identifies the Food Container Set (104) because every 2025 price is unique; log it. (b) **Quarantine**: revenue needs quantity, and it can't be recovered from the export. (c) **Keep and label** "(unassigned)": revenue doesn't need it. (d) **Standardize** to NULL, then keep: the customer's orders still count; city reports show an "unknown" group.
6. MySQL's default collation (utf8mb4_0900_ai_ci) is case-insensitive, so Delivered , DELIVERED , and delivered fall into one group (21,185 rows instead of 19,835 for the exact spelling). Trailing-space variants stay separate. Use GROUP BY BINARY status (or COLLATE utf8mb4_bin) to see every spelling.
7. SELECT branch, COUNT(*) FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+\$' GROUP BY 1 returns 17 spellings; COUNT(DISTINCT LOWER(TRIM(branch))) returns 12: mumbai ho , mumbai h.o. , mumbai → Mumbai HO; bengaluru , bangalore , blr → Bengaluru; delhi , new delhi , del → Delhi; kolkata , calcutta , kol → Kolkata. (mumbai meaning the Mumbai HO branch is a business rule to confirm, since a customer's city can also be Mumbai.)
8. SELECT COUNT(*), COUNT(DISTINCT stg_orders_raw.*), COUNT(DISTINCT order_item_id) FROM stg_orders_raw WHERE order_item_id ~ '^[0-9]+\$' returns 25,969, 25,832, 25,832. Distinct whole rows equal distinct IDs, so every repeated ID is an identical copy: 137 duplicate rows. MySQL: SELECT COUNT(*) FROM (SELECT DISTINCT * FROM stg_orders_raw WHERE order_item_id REGEXP '^[0-9]+\$') x gives 25,832.
9. In clean_order_lines , WHERE date_repaired (the query in section 14.7) lists 9 lines with exported dates 31-09-2025 , 31-11-2025 , or 32-10-2025 . Each was repaired from the entry timestamp converted to IST. In Power Query, the try ... otherwise null column returns null for those 9 rows; filter for null to list them.
10. 40 (regexp_replace(order_date, '[0-9]', '9', 'g') = '99999'). SQL: DATE '1899-12-30' + 45940 → 2025-10-10 (MySQL: DATE '1899-12-30' + INTERVAL 45940 DAY). Excel: type 45940 in a cell and format it as a date. Power Query: Date.From(45940) .
11. Before: 25,299 of 25,832 lines match (97.9%); after LPAD(TRIM(code), 4, '0') : 25,832 (100%). All 533 failures are Kolkata lines (spelled Kolkata, Calcutta, kolkata, or KOL in the raw data), whose spreadsheet upload drops leading zeros. Only 533 of Kolkata's 3,210 lines fail because codes of 1000 and above have no leading zero to lose.
12. Naive: Rs. 750 → .750 → 0.75; ₹750.00 → 750.00 → 750. Correct: regexp_replace(regexp_replace(unit_price, '^[^0-9]+', ''), ',', '', 'g')::numeric (MySQL: CAST(REPLACE(REGEXP_REPLACE(unit_price, '^[^0-9]+', ''), ',', '') AS DECIMAL(10,2))). Rule: SELECT COUNT(*) FROM clean_order_lines c JOIN products p USING (product_id) WHERE c.unit_price <> p.unit_price must return 0; the naive version would fail on every Rs. line.
13. 1,362 lines, all from Kolkata (the smallest non-zero discount there is 0.05). Rule: CASE WHEN d > 0 AND d < 1 THEN d * 100 ELSE d END . It's unsafe wherever real discounts below 1% exist (a 0.5% early-payment discount would become 50%); then you need a source-system flag or documentation, not a

value rule.

14. 141 lines with `qty_unit = 'CTN' : 540` as written, **5,400** pieces after $\times 10$.

15. Six lines: 187761 (700), 200981 (450), 206224 (450), 191340 (400), 205978 (350), 209166 (100). As written they total **₹1,219,750** of non-cancelled net revenue; with quantities divided by 10 they'd be ₹121,975, so leaving them in would overstate Q4 by **₹1,097,775**. (They're quarantined, not "fixed", until the branches confirm.)

16. 1,598 lines have a different date in UTC (section 14.7). A different month: `COUNT(*) FILTER (WHERE date_trunc('month', entered_at_ist - INTERVAL '5 hours 30 minutes') <> date_trunc('month', order_date::timestamp))` returns **43**: web entries shortly after midnight IST on the 1st of a month, which fall on the last day of the previous month in UTC. A monthly report grouped by UTC would move them to the wrong month.

17. `SELECT COUNT(*) FROM clean_order_lines c WHERE sales_rep IS NOT NULL AND NOT EXISTS (SELECT 1 FROM employees e WHERE e.employee_name = c.sales_rep)` returns **0**. In the raw (deduplicated) data, **875** rows had a trailing space in `sales_rep`; without `TRIM`, all 875 would fail the rule.

18. 25,832 lines exist in both; **25,812** are identical. The 20 differences are the quarantined lines: 14 with no quantity (`NULL` never equals a number) and 6 with quantities ten times too large.

19. After `LOWER(TRIM())` and `map_segment`: **Retail 2,791, Hospitality 1,513, Wholesale 723** (5,027). These still include the 48 duplicate records.

20. Name key only: **48** groups; name key plus city: **46**. In two duplicate pairs, one record has a city and the other doesn't, and `NULL` never equals a value, so the stricter rule misses them. A stricter rule protects against merging different businesses with the same name in different cities; here it costs two missed duplicates and prevents no false matches. Measure both errors when you can, and treat `city IS NULL` explicitly (for example, match when cities are equal or either is missing).

21. Text `NULL` **24**, N/A **22**, - **20**, unknown **19**, real blank **15**: **100** records with no usable city.

22. Top 1,000 rows: **8** distinct statuses (the first rows come from the October part of the file, where not every spelling has appeared yet). Entire dataset: **18** (plus the header text if you haven't removed non-data rows). The difference is why profiling must cover the whole file.

23. 7: delivered, pending, shipped, cancelled, dlvd, cxl, canceled.

24. Quick way: Mumbai HO ₹132,385,776; Delhi ₹101,831,086; Bengaluru ₹101,788,506; Kolkata ₹54,539,528 (total ₹390,544,896). Clean (excluding cancelled and quarantined): Mumbai HO ₹150,854,187; Bengaluru ₹117,619,062; Delhi ₹102,966,828; Kolkata ₹52,120,934 (total ₹423,561,011). **Delhi and Bengaluru** swap places.

25. `SELECT unit_price, COUNT(*) FROM products GROUP BY unit_price HAVING COUNT(*) > 1` returns no rows (and `COUNT(DISTINCT unit_price)` is 8 for 8 products); to prove it for 2025 prices actually charged, group `order_items` for 2025 by `unit_price` and count distinct `product_id`s, which is 1 for every price. If two products shared a price in 2026, the repair would be ambiguous: quarantine those lines, or use another reliable column (a product description, the order's other lines, or the ERP's audit log).

26. `similarity('Balaji Distributors','BALAJI DISTRIBUTORS') = 1` (pg_trgm ignores case); `'Balaji Distrubutors' = 0.73913044`; `'Balaji Traders' = 0.2962963`. A review threshold around 0.6–0.7, within the same city, catches typos while ignoring names that share only a common word. Tune it by reviewing a sample above and below the threshold.

27. `SELECT customer_code, customer_name, signup_date FROM clean_customers WHERE signup_in_future`: **0772** Ganesh Supermarket Bhubaneswar (2060-05-03), **1228** Jai Hind Provisions Hyderabad (2061-08-22), **1380** Nandi Bazaar Gurugram (2064-02-25), **3058** Bharat Stores Kochi (2065-01-21), **3661** New Depot (2065-06-21). A message: *"Five customer records have signup dates in the 2060s, which look like typing slips (for example 2064 for 2024). List attached with codes and names. Could the CRM team correct them from the account-opening forms, and add a rule that rejects signup dates after today?"*

28. Explain the risk and offer an alternative: filling with the average (38 pieces) invents 14 sales that may be larger, smaller, or not real, and the report would look complete while being wrong in ways nobody can see. Instead, report revenue excluding the 14 lines, state their count and that quantities are being confirmed with the branches, and update the report once they're confirmed. If a single estimate is needed for planning, show it separately and label it as an estimate.

29. Changing the system of record as part of an analysis is dangerous: a match key finds **candidates**, and a wrong merge moves order history, credit terms, and contacts between businesses. `DELETE` also breaks orders that point at the deleted customer. Instead: produce a review list (both records side by side with their orders and contacts), agree a merge rule with the CRM owner, have a person confirm each pair, and use the CRM's own merge feature so history moves with the record. For analysis in the meantime, group duplicates with the match key in your query.

30. A reasonable answer: normalize email (trim, lower case) and phone (digits only, with country code) and use them as the primary keys; for leads without either, use a match key on company name (case, spaces, legal suffix) plus city, and a person's name. Keep the source and date of each record so the earliest or richest record survives. For leads, it's usually better to **miss** some duplicates (a salesperson may call the same person twice) than to **merge** different people (losing a real prospect's details or attributing one person's consent to another). Measure the error on a hand-checked sample before applying the rule to all 40,000.

Timed challenge answers. Level 1: **357** exactly `Mumbai` ; **409** after cleaning. Level 2: **64** distinct values as exported; **39** cities after cleaning. Level 3: **Retail 2,791, Hospitality 1,513, Wholesale 723**. Level 4: **100** with no usable city; **150** missing emails; **58** without `@` . Level 5: **400** dates as `DD/MM/YYYY` ; **5** after 2025, earliest **2060-05-03**. Level 6: **48** duplicate groups; **4,979** businesses. Level 7: **Retail 2,756, Hospitality 1,505, Wholesale 718; 405** in Mumbai. Bonus: **88**.